

Нейросетевое ранжирование II

ШАД, Рекомендательные системы
2025/26, лекция 7

Кирилл Хрыльченко

Старший преподаватель,
ФКН ВШЭ

План занятия

Продолжаем обсуждение нейросетевого ранжирования:

1. Feature interaction modelling
2. MLP layers
3. Multi-task learning
4. Position debiasing
5. Knowledge distillation

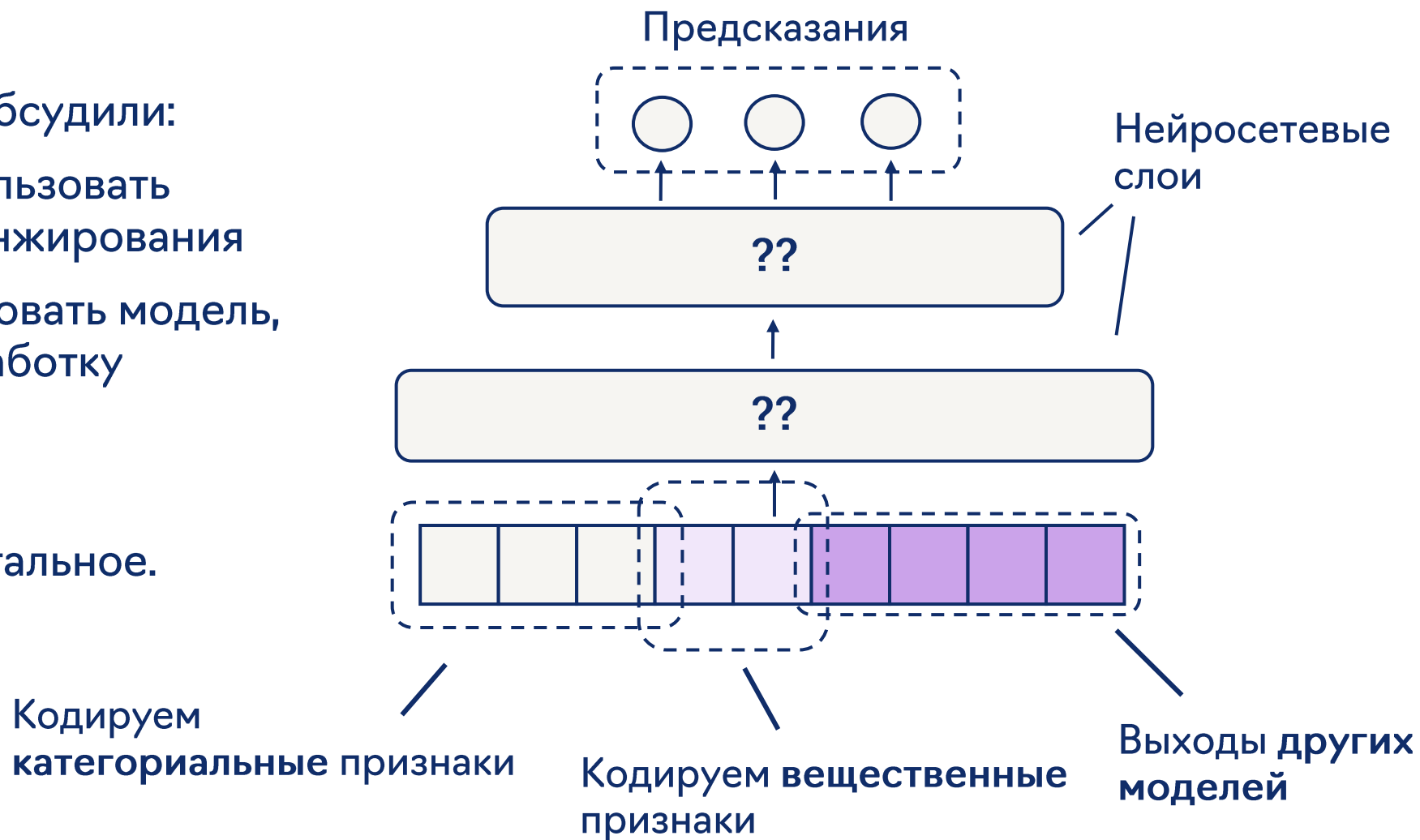
На семинаре – разбор статьи про DCN-v2.

Recap

На прошлой лекции обсудили:

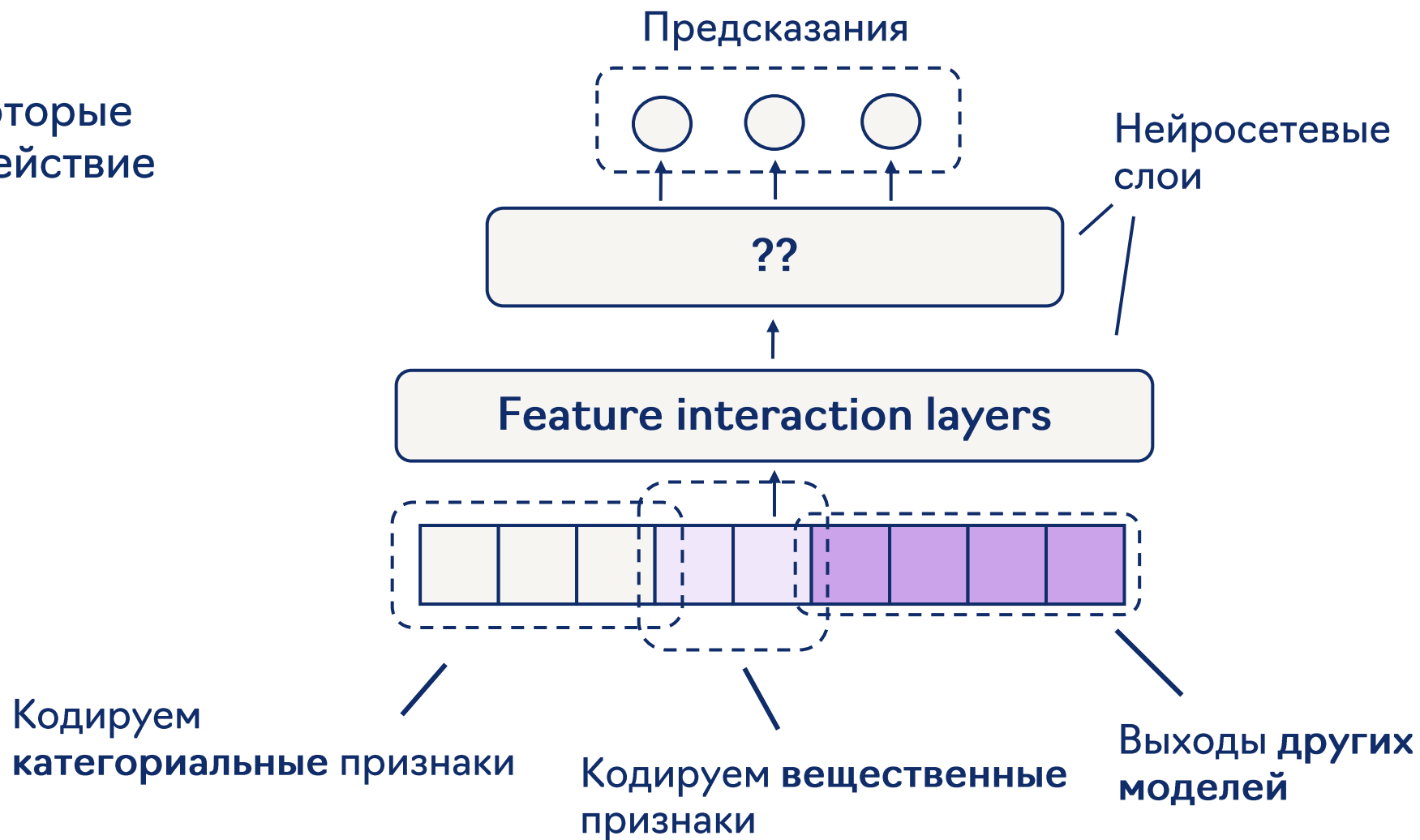
- когда нужно использовать нейросети для ранжирования
- начали конструировать модель, рассмотрели обработку признаков

Сегодня – про всё остальное.



Feature interaction layers

Специальные слои, которые моделируют взаимодействие признаков.



Modeling feature interactions

Часть 1

Линейная модель

$$y = \langle w, x \rangle + b,$$

- $w \in \mathbb{R}^n, b \in \mathbb{R}$ – параметры модели
- На входе конкатенация one-hot представлений категориальных признаков:

$$x = [x_1 \mid \dots \mid x_m], x_i \in \mathbb{R}^{n_i}, \sum_{i=1}^m n_i = n$$

- Это равносильно тому, что мы:
 - Каждому значению признака сопоставили один обучаемый скаляр
 - Складываем скаляры реализовавшихся значений признаков

Кросс-признаки

$$\phi(x) = \prod_{i=1}^n x_i^{c_i}, \quad c_i \in \{0, 1\}$$

- Комбинируем признаки
 - E.g., “*language=ru, genre=rock*”
- Добавим все **кросс-признаки** второй степени:

$$y = \sum_{i=1}^n w_i x_i + \sum_{i>j} W_{ij} x_i x_j + b$$

- Возникают проблемы:
 - Очень большая матрица $W \in \mathbb{R}^{n \times n}$
 - Плохая обобщающая способность – не умеем работать на unseen значениях кросс-признаков

Factorization machines

- Сделаем матричную факторизацию:

$$W = \text{Lower}(VV^T), V \in \mathbb{R}^{n \times d}$$

- Тогда наша модель:

$$y = \sum_{i=1}^n w_i x_i + \sum_{i>j} \langle v_i, v_j \rangle x_i x_j + b$$

- Матрица V – эмбединги признаков
- Решили обе проблемы:
 - Сложность по памяти $O(nd)$
 - Обобщаемся на unseen комбинации признаков

Field-aware factorization machines

$$y = \sum_{i=1}^n w_i x_i + \sum_{i>j} \langle v_i^j, v_j^i \rangle x_i x_j + b$$

- Для каждой пары признаков используем свои эмбединги
- Победили в соревновании Kaggle Criteo CTR Prediction Challenge (2014):
 - Команда называлась “3 idiots”
 - Позже победили в соревновании Kaggle Avazu CTR Prediction Challenge

Linear vs Poly2 vs FM vs FFM

	Model	time per epoch (seconds)	leaderboard logloss	rank
Criteo	Linear	40	0.46224	91
	Poly2	1,206	0.44956	14
	FM	446	0.44818	10
	FFM	731	0.44603	3
	FFM*	-	0.44464	1
Avazu	Linear	16	0.38833	64
	Poly2	91	0.38347	10
	FM	71	0.38407	11
	FFM	85	0.38223	6
	FFM*	-	0.37913	1

FFM*: FFM with ensemble setting



23 / 29

[RecSys 2016: Paper Session 2 - Field Aware Factorization Machines for CTR Prediction](#)

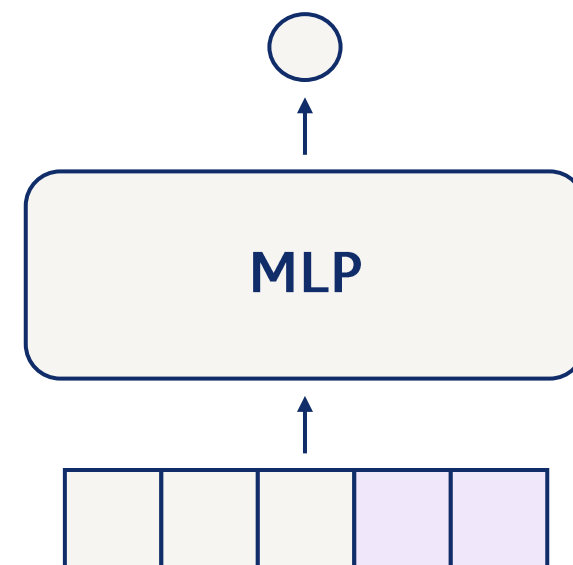
Concat + MLP

Простейшая нейросеть для ранжирования:

- Конкатенируем:
 - обучаемые эмбединги категориальных признаков
 - нормализованные вещественные признаки
- Применяем MLP:

$$x_{l+1} = f(W_l x_l + b_l)$$

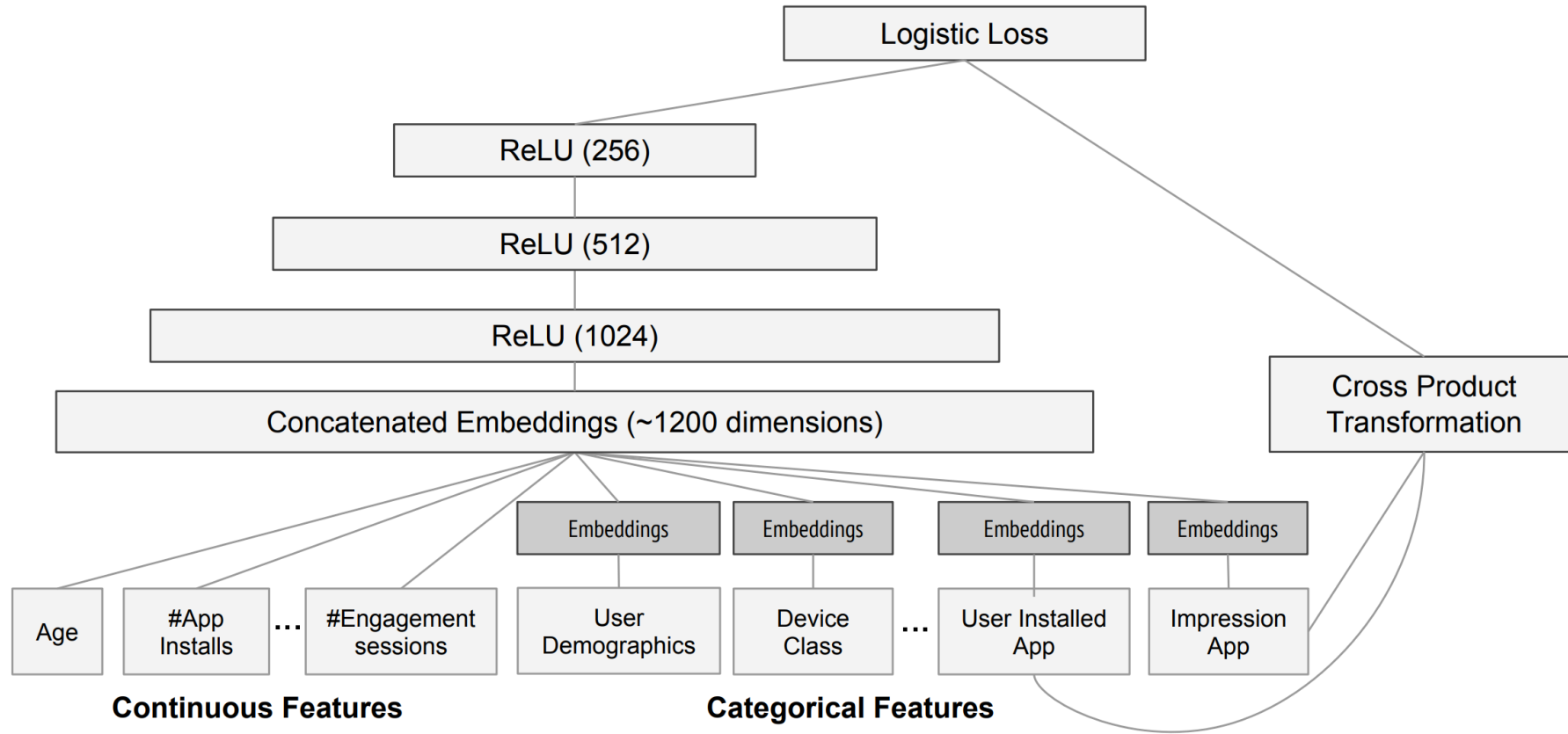
где f – функция активации, l – номер слоя



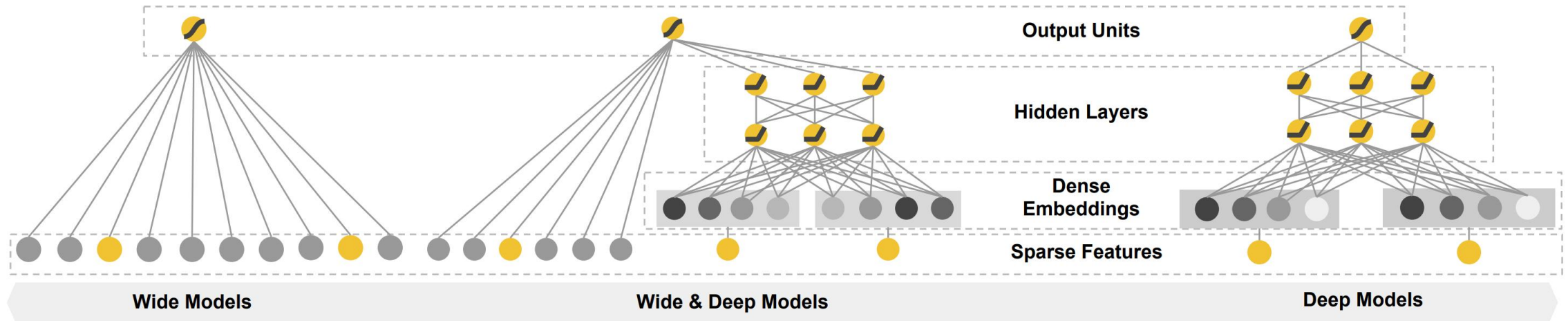
Wide & Deep

- Была линейная модель с вручную отобранными кросс-признаками
- Добавили нейросеть:
 - Эмбединги размера 32 категориальных признаков
 - $F(X \leq x)$ преобразование вещественных признаков
 - Конкатенация – вектор размерности 1200
 - 3 слоя с ReLU
- Финальное предсказание – сумма предсказаний линейной модели и нейросети

Wide & Deep



Wide & Deep



Wide & Deep

Линейная модель – **wide** часть:

- Меморизация
- Явно моделирует взаимодействия между признаками до второго порядка

Нейросеть – **deep** часть:

- Обобщающая способность
- Неявно моделирует взаимодействия высокого порядка

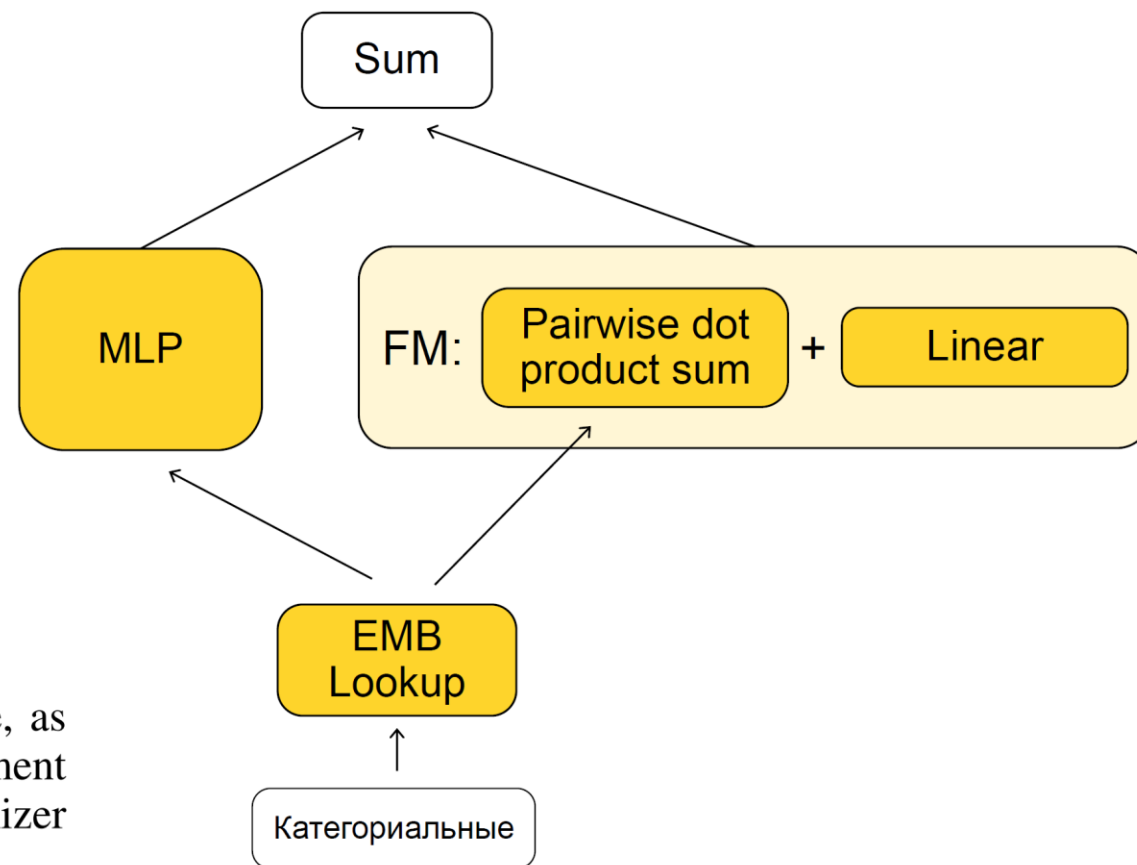
Table 1: Offline & online metrics of different models. Online Acquisition Gain is relative to the control.

Model	Offline AUC	Online Acquisition Gain
Wide (control)	0.726	0%
Deep	0.722	+2.9%
Wide & Deep	0.728	+3.9%

DeepFM

- Заменяли ручные кросс-признаки на факторизационную машину
 - Чтобы не заниматься feature engineering
- Эмбединги признаков используются и в нейросети, и в FM

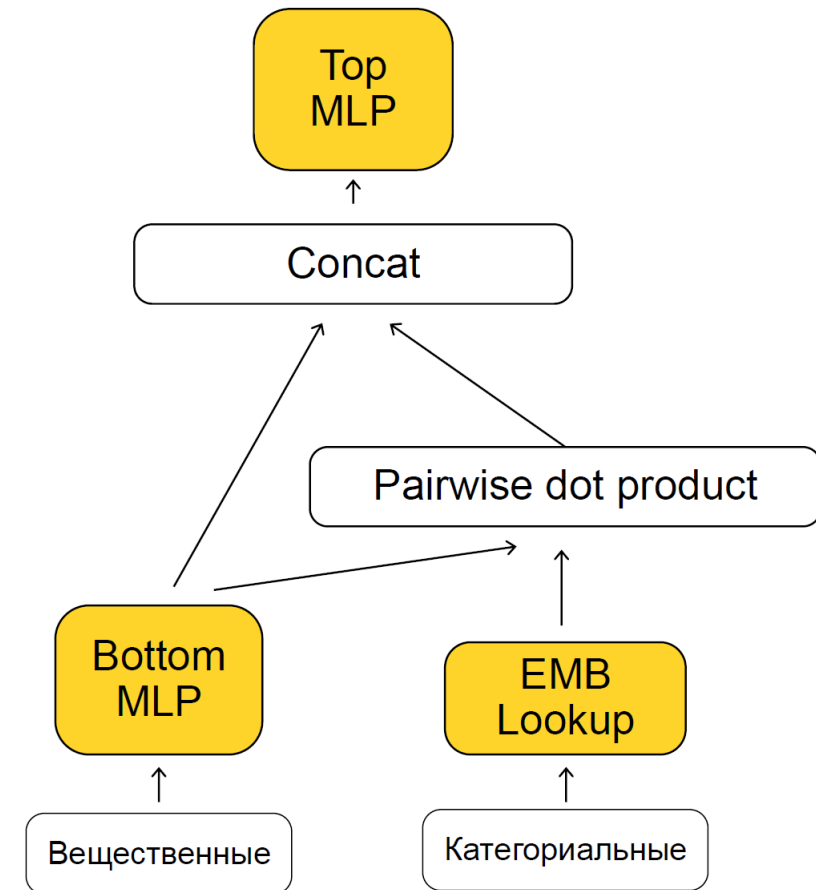
⁶We do not use the Wide & Deep API released by Google, as the efficiency of that implementation is very low. We implement Wide & Deep by ourselves by simplifying it with shared optimizer for both deep and wide part.



DLRM

- **Bottom MLP:** превращаем вещественные признаки в единый эмбединг
- **Top MLP:** конкатенируем попарные скалярные произведения и выход bottom MLP, применяем MLP
 - В попарных скалярных произведениях участвует выход bottom MLP

А как сделать модель, у которой разные размерности эмбедингов у признаков?

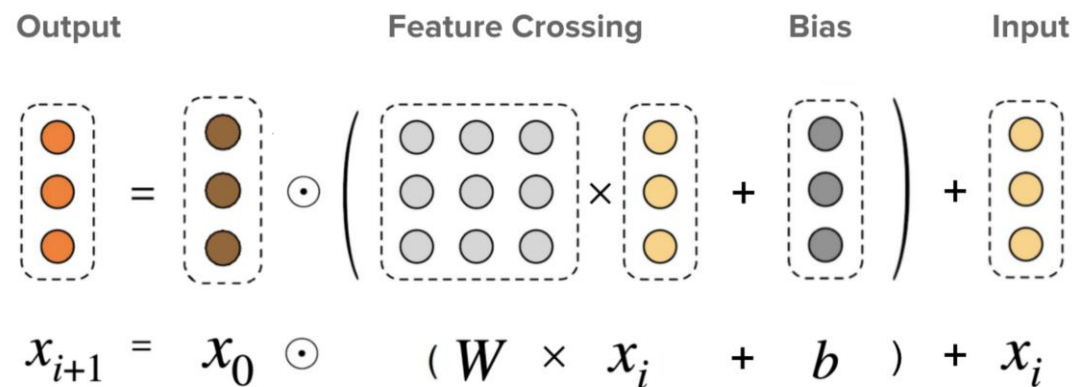


DCN-v2

Кросс-слой:

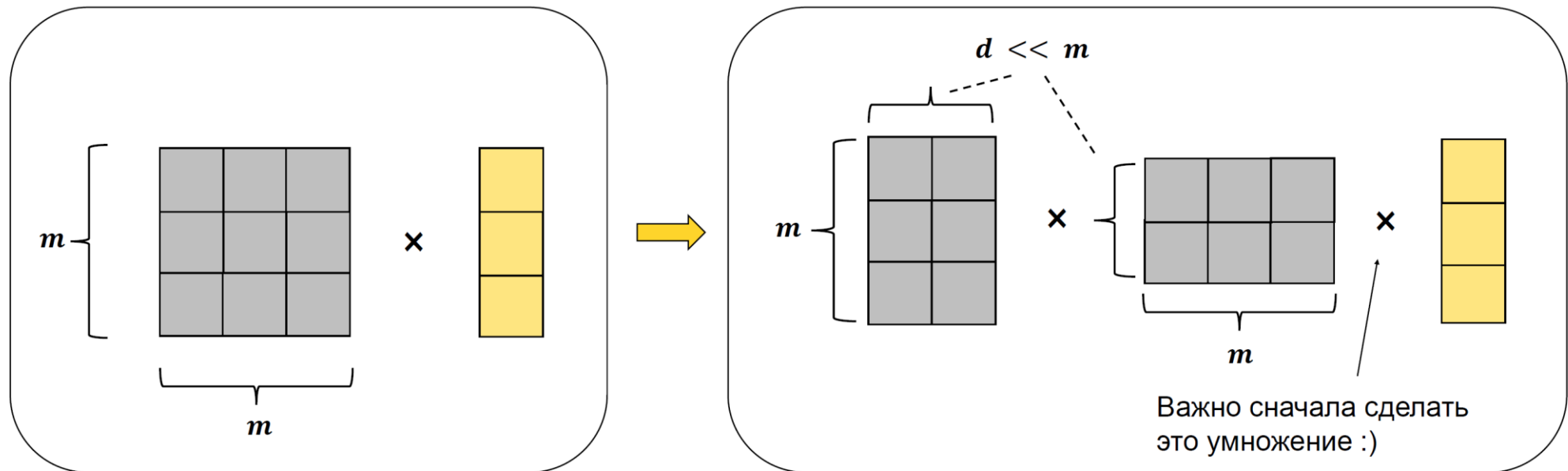
$$x_{l+1} = x_0 \odot (W_l x_l + b_l) + x_l$$

- Можно получить скалярное произведение любых векторов внутри конкатенации
- L слоев моделируют всевозможные комбинации признаков степени L + 1
- Diminishing returns: больше 2-3 слоёв не даёт прирост качества
- Это вычислительно очень тяжелый слой



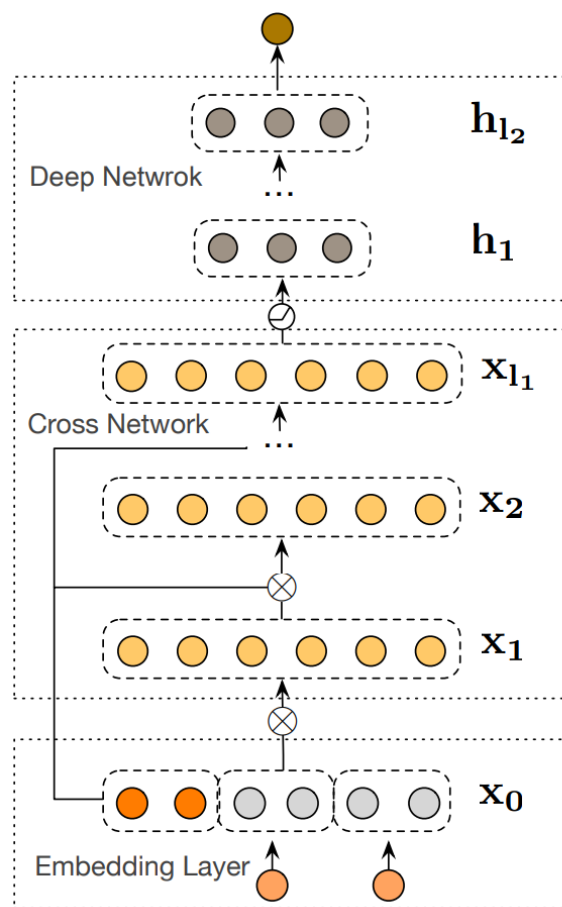
Low-rank cross layer

Сделаем матричную факторизацию:

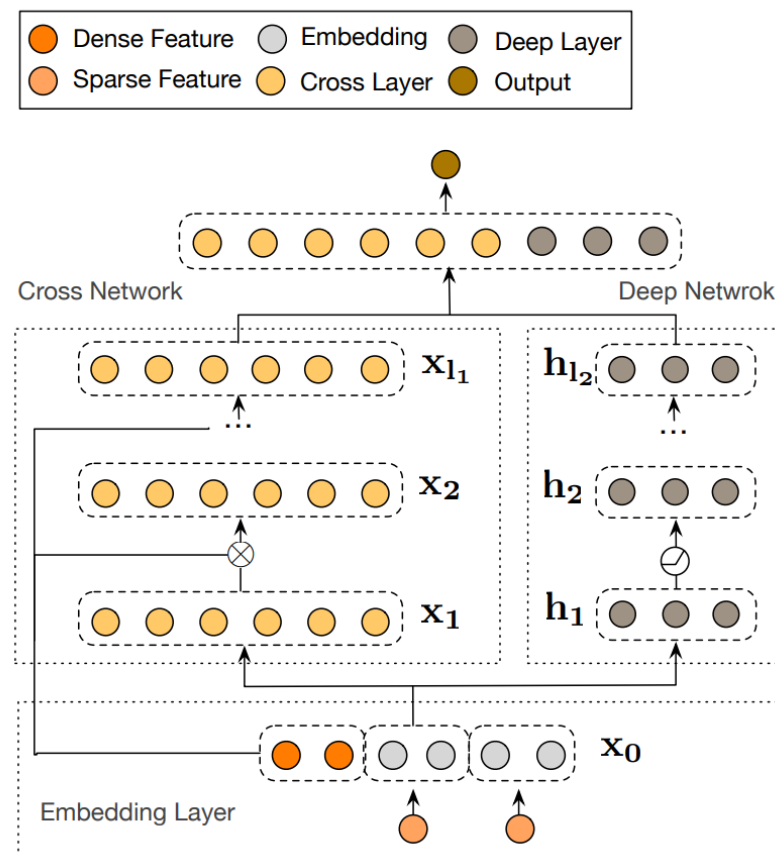


А еще – можно сделать смесь таких низкоранговых разложений.

Deep and cross combination



(a) Stacked



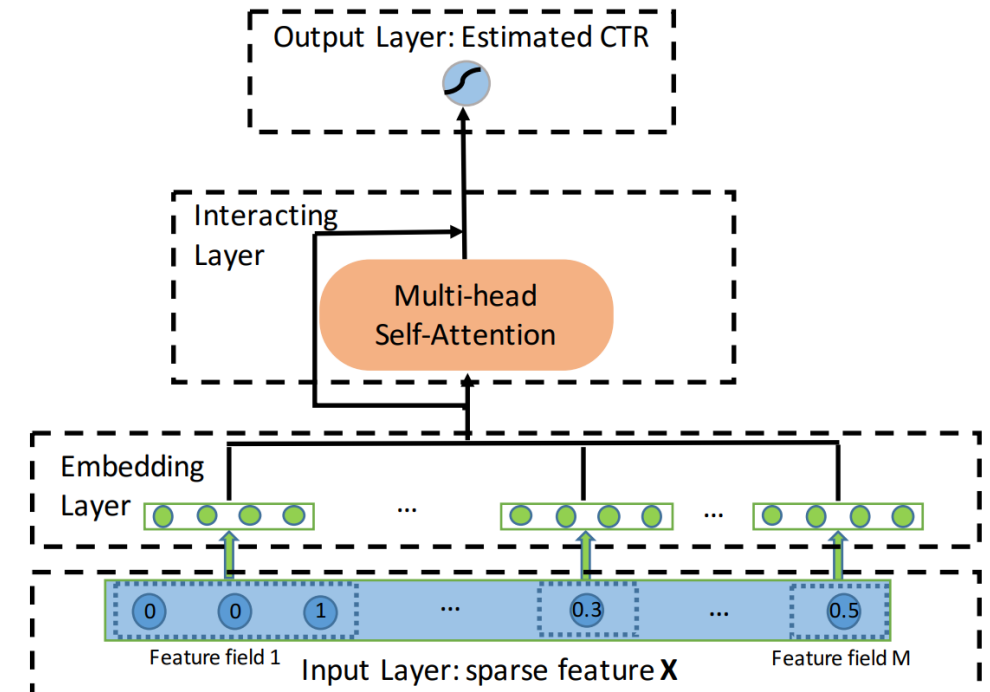
(b) Parallel

Deep and cross combination

- Cross network комбинируют с MLP:
 - Parallel vs stacked; stacked обычно работает лучше
 - Перед MLP широкий вектор из cross network сужается
- Cross network не дает приростов поверх абстрактных семантических векторных представлений
 - его нужно применять именно над конкатенацией векторов признаков
- Cross network явно моделирует взаимодействие признаков низкого порядка
 - а MLP – неявно и высокого порядка

AutoInt

- **Обработка входов:**
 - Обучаемые эмбединги категориальных признаков
 - Вещественные признаки домножаем на обучаемый эмбединг
- Применяем multi-head attention со skip connection
- MLP поверх конкатенированных векторов на выходе attention



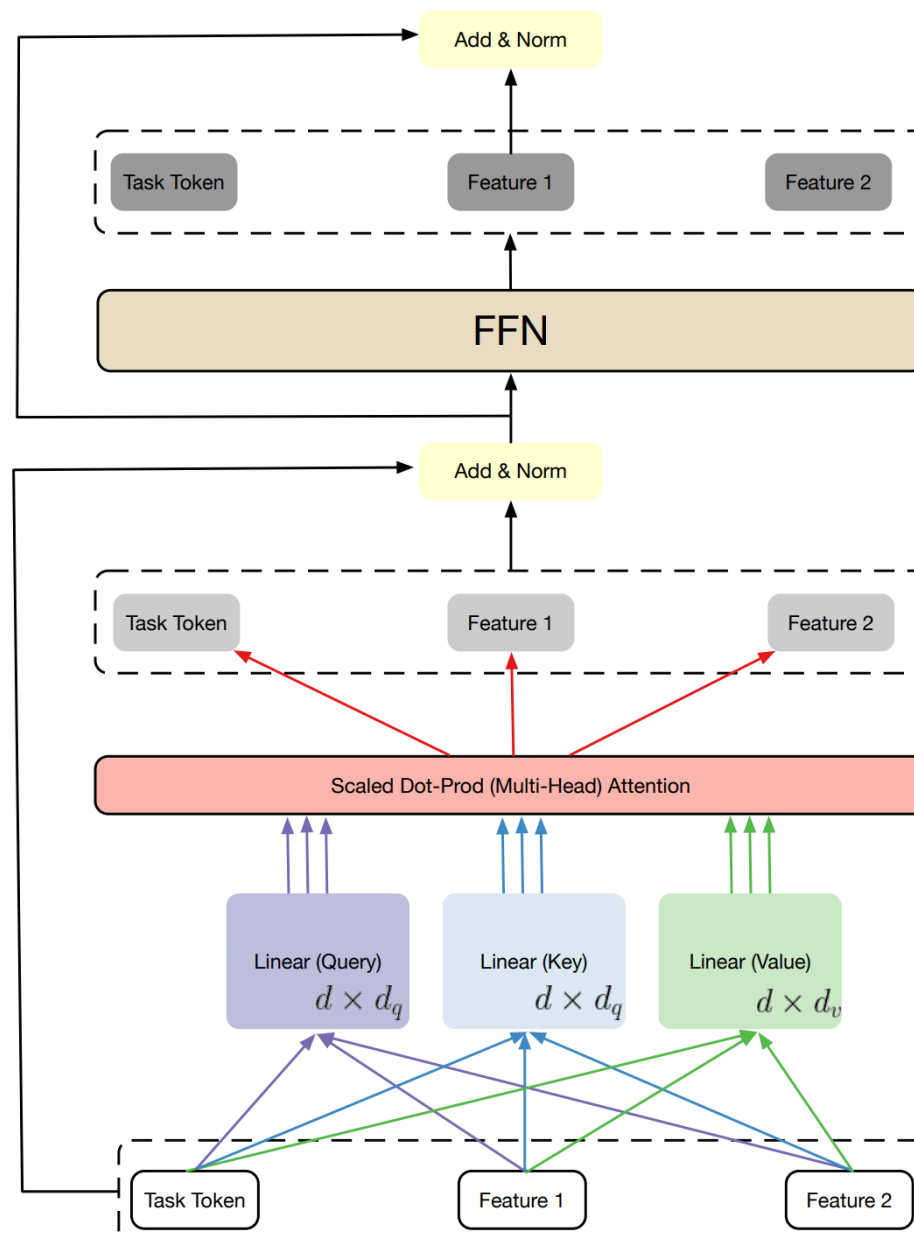
Работает хуже, чем DCN-v2.

Hiformer

Трансформер одинаково обрабатывает все признаки (и их комбинации)

- Это хорошо работает для неструктурированных данных (e.g., тексты), потому что они **однородные**
- А в рекомендациях – признаки имеют разную природу, данные **гетерогенные**

Нужно обрабатывать разные признаки по-разному.



Гетерогенный attention

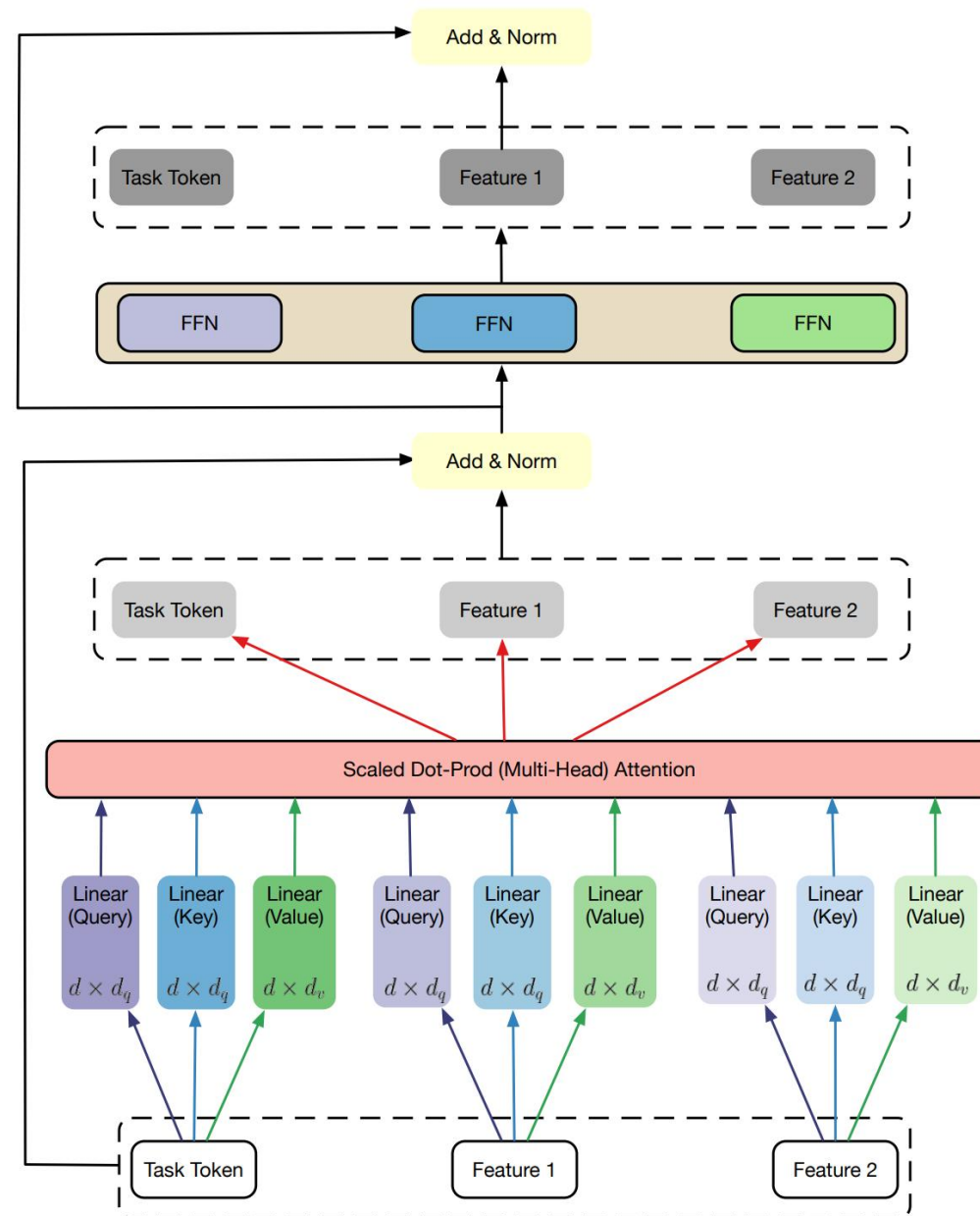
- Раздельные Q,K,V для каждого признака
- FFN слой тоже свой для каждого признака

$$\text{Att}(i, j)^h = \frac{\exp(\phi_{i,j}^h(\mathbf{e}_i, \mathbf{e}_j))}{\sum_{m=1}^L \exp(\phi_{i,m}^h(\mathbf{e}_i, \mathbf{e}_m))}$$

$$\phi_{i,j}^h(\mathbf{e}_i, \mathbf{e}_j) = \frac{\mathbf{e}_i \mathbf{Q}_i^h (\mathbf{e}_j \mathbf{K}_j^h)^T}{\sqrt{d_k}}$$

Такие слои требуют очень много памяти.

[Hiformer: Heterogeneous Feature Interactions](#)
[Learning with Transformers for Recommender Systems](#)



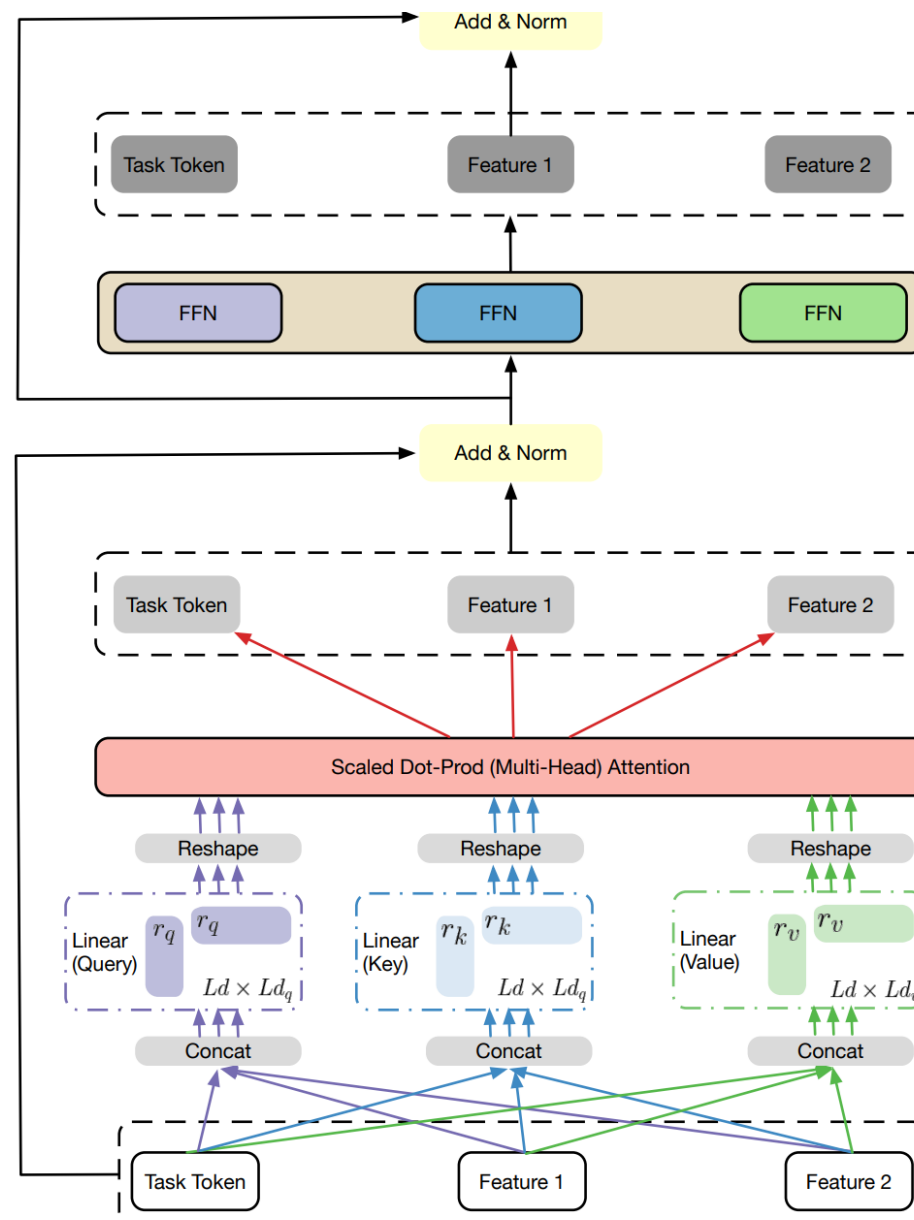
Hiformer

Композитные признаки: конкатенируем векторы признаков, умножаем на матрицу, выход делим на токены

$$[\hat{\mathbf{k}}_1^h, \dots, \hat{\mathbf{k}}_L^h] = \text{concat}([\mathbf{e}_1^h, \dots, \mathbf{e}_L^h])\hat{\mathbf{K}}^h$$

Это вычислительно тяжелый слой

[Hiformer: Heterogeneous Feature Interactions
Learning with Transformers for Recommender Systems](#)



Hiformer

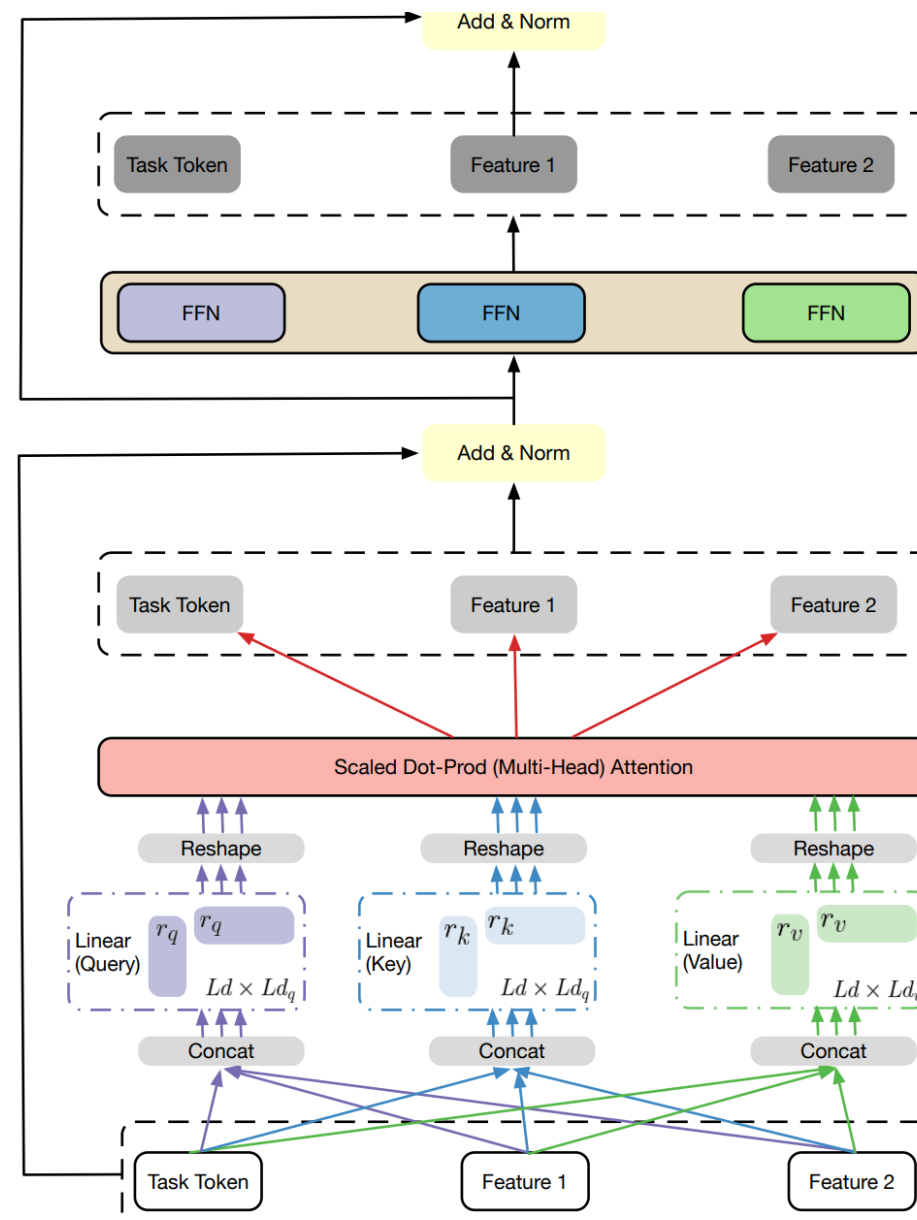
Композитные признаки: конкатенируем векторы признаков, умножаем на матрицу, выход делим на токены

$$[\hat{\mathbf{k}}_1^h, \dots, \hat{\mathbf{k}}_L^h] = \text{concat}([\mathbf{e}_1^h, \dots, \mathbf{e}_L^h])\hat{\mathbf{K}}^h$$

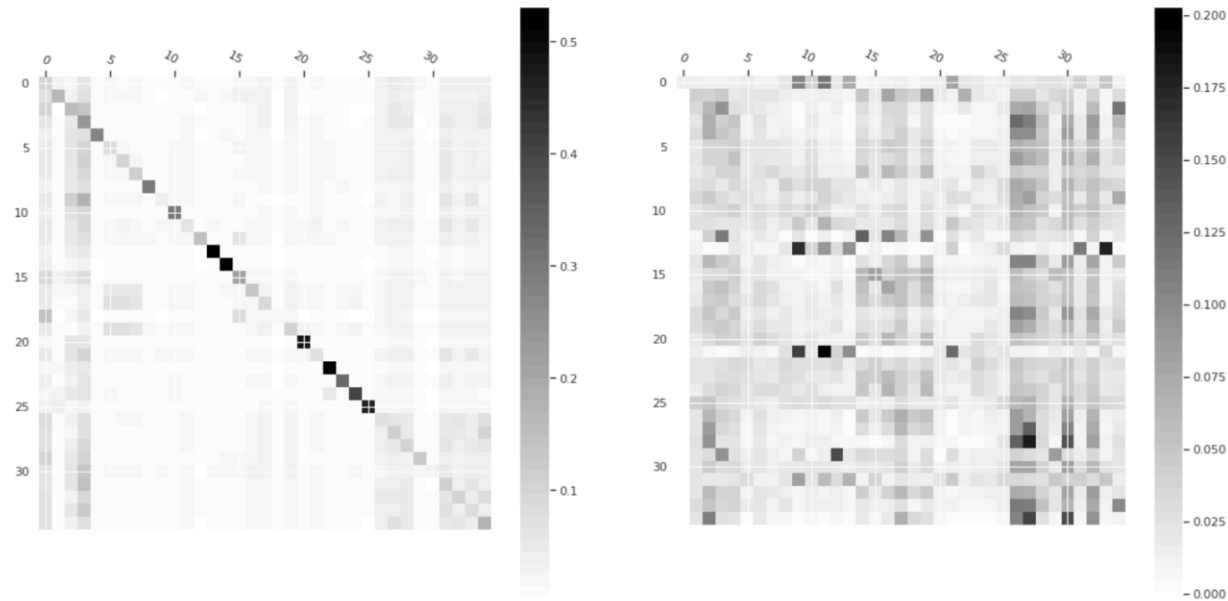
Это вычислительно тяжелый слой

- Сделаем матричную факторизацию

[Hiformer: Heterogeneous Feature Interactions
Learning with Transformers for Recommender Systems](#)



Hiformer



(a) Vanilla Transformer Attention Layer. (b) Heterogeneous Attention Layer.

Figure 2: Attention patterns with feature interaction layers.

Field-aware transformer

- Сделали для каждой пары признаков свой обучаемый коэффициент внутри attention

$$\alpha(i, j) = (\mathbf{q}_i^\top \mathbf{k}_j) \cdot w_{f_i, f_j},$$

$$\text{where } \mathbf{q}_i = \mathbf{W}_Q^{(f_i)} \mathbf{h}_i, \quad \mathbf{k}_j = \mathbf{W}_K^{(f_j)} \mathbf{h}_j$$

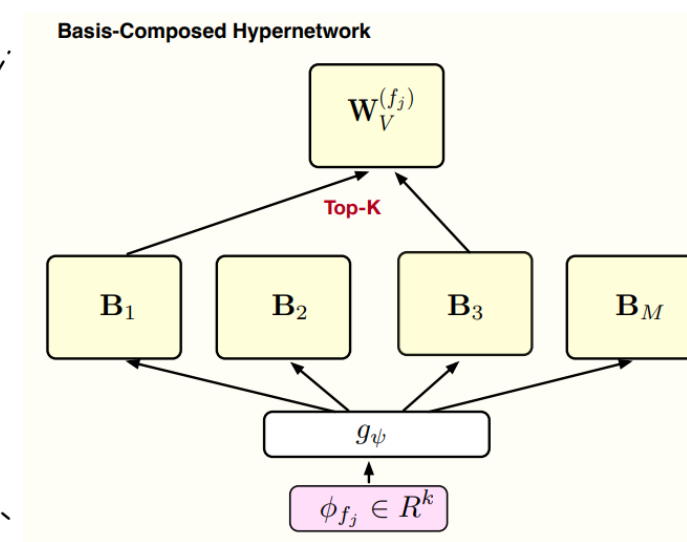
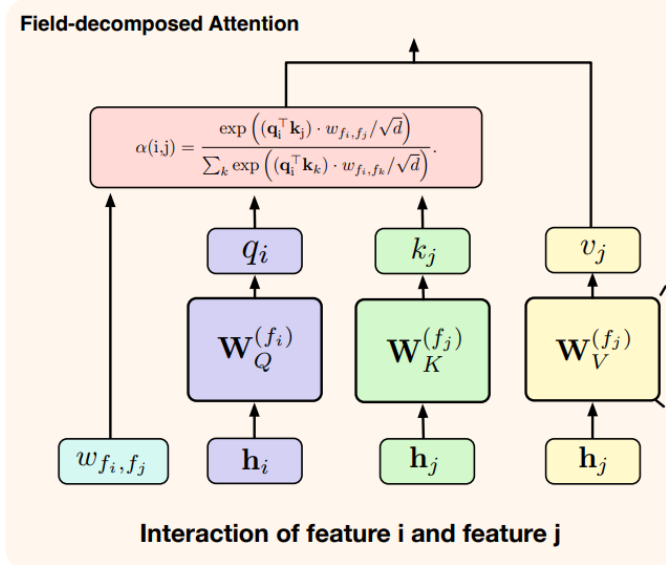
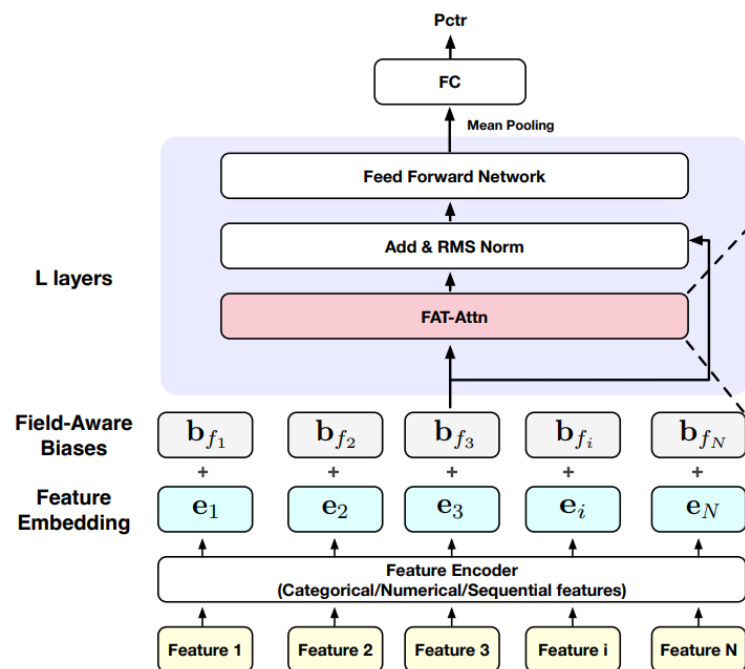
- Вместо Q, K, V используют смесь матриц (у каждого признака свои веса смеси)

$$\mathbf{W}_Q^{(f)} = \sum_{m \in \pi_f} \alpha_m^{(f)} B_m$$

$$\text{FAT-Attn}_h(i) = \sum_j \alpha_h(i, j) \mathbf{v}_j^{(h)},$$

$$\alpha_h(i, j) = \frac{\exp \left((\mathbf{q}_{i,h}^\top \mathbf{k}_{j,h}) \cdot w_{f_i, f_j}^{(h)} / \sqrt{d} \right)}{\sum_k \exp \left((\mathbf{q}_{i,h}^\top \mathbf{k}_{k,h}) \cdot w_{f_i, f_k}^{(h)} / \sqrt{d} \right)}.$$

Field-aware transformer



RankMixer: token mixing

Заменили attention на простое смешивание векторов токенов:

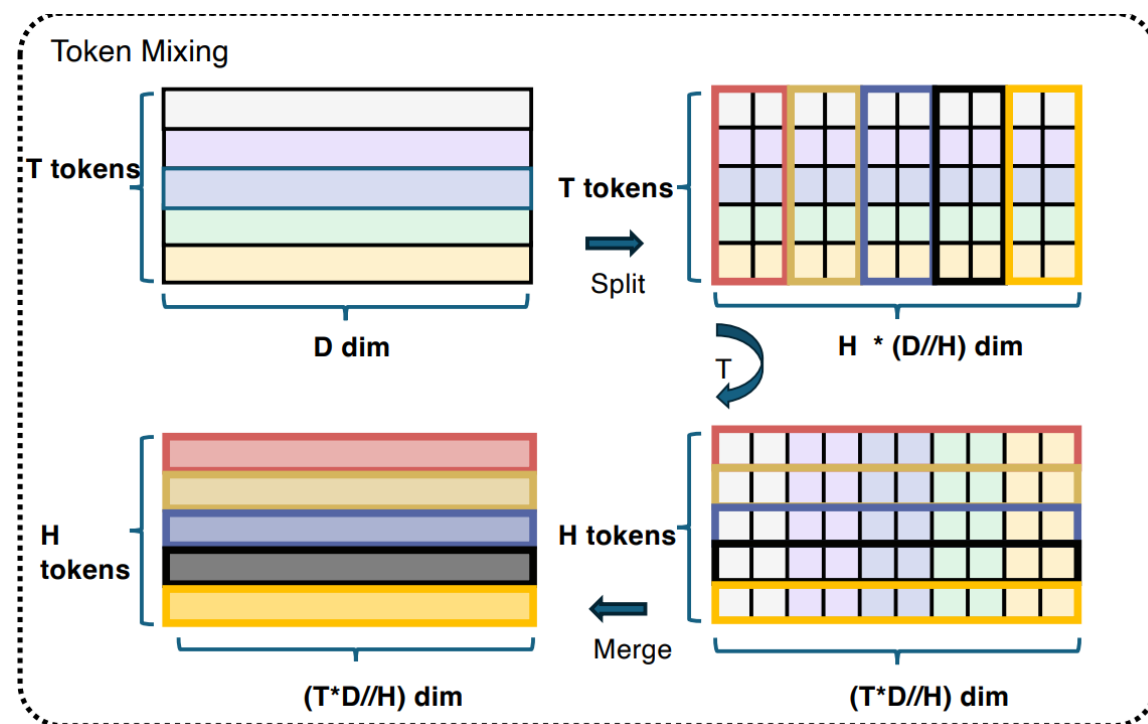
- Делим вектор каждого токена на H частей

$$\left[\mathbf{x}_t^{(1)} \parallel \mathbf{x}_t^{(2)} \parallel \dots \parallel \mathbf{x}_t^{(H)} \right] = \text{SplitHead}(\mathbf{x}_t)$$

- Склеиваем соответствующие кусочки всех токенов

$$\mathbf{s}^h = \text{Concat}(\mathbf{x}_1^h, \mathbf{x}_2^h, \dots, \mathbf{x}_T^h)$$

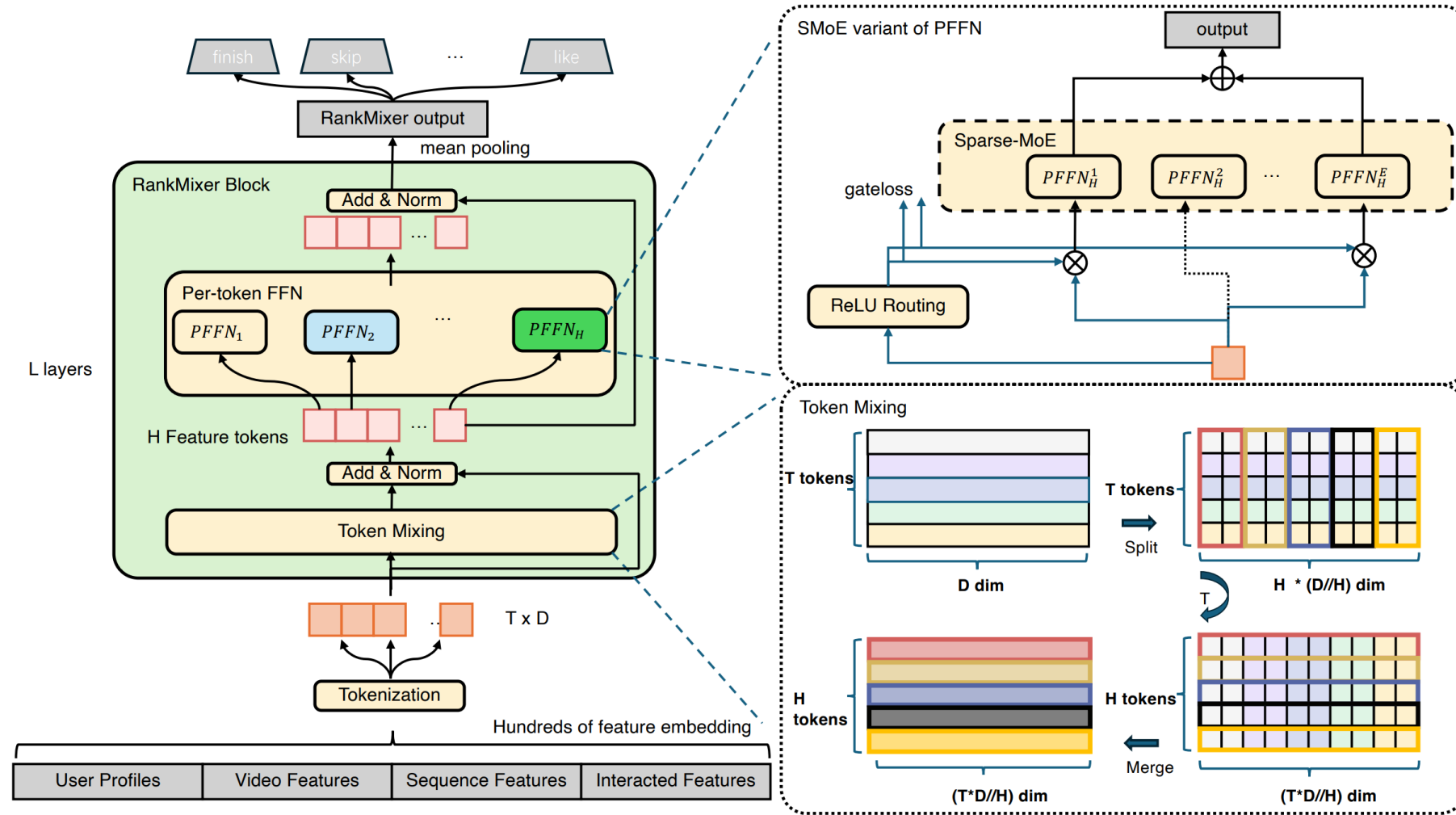
- Фиксируется $H = T$
 - во второй версии модели это исправили



[RankMixer: Scaling Up Ranking Models in Industrial Recommenders](#)

[TokenMixer-Large: Scaling Up Large Ranking Models in Industrial Recommenders](#)

Per-token FFN



State of the art for feature interaction modeling

Table 2: Performance & efficiency comparison of ~500 M-parameter models on E-commerce (best values in bold).

Model	CTCVR	Efficiency	
	$\Delta\text{AUC}\uparrow$	Params	FLOPs/Batch
DLRM-MLP-500M [24]	–	499 M	125.1 T
HiFormer [12]	+0.44%	570 M	28.8 T
DCNv2 [31]	+0.49%	502 M	125.8 T
DHEN [37]	+0.63%	415 M	103.4 T
AutoInt [25]	+0.75%	549 M	138.6 T
Wukong [36]	+0.76%	513 M	4.6 T
Group Transformer	+0.81%	550 M	4.5 T
FAT [34]	+0.82%	551 M	4.59 T
RankMixer(TokenMixer) [40]	+0.84%	567 M	4.6 T
TokenMixer-Large 500M	+0.94%	501 M	4.2 T

Выводы

Обсудили:

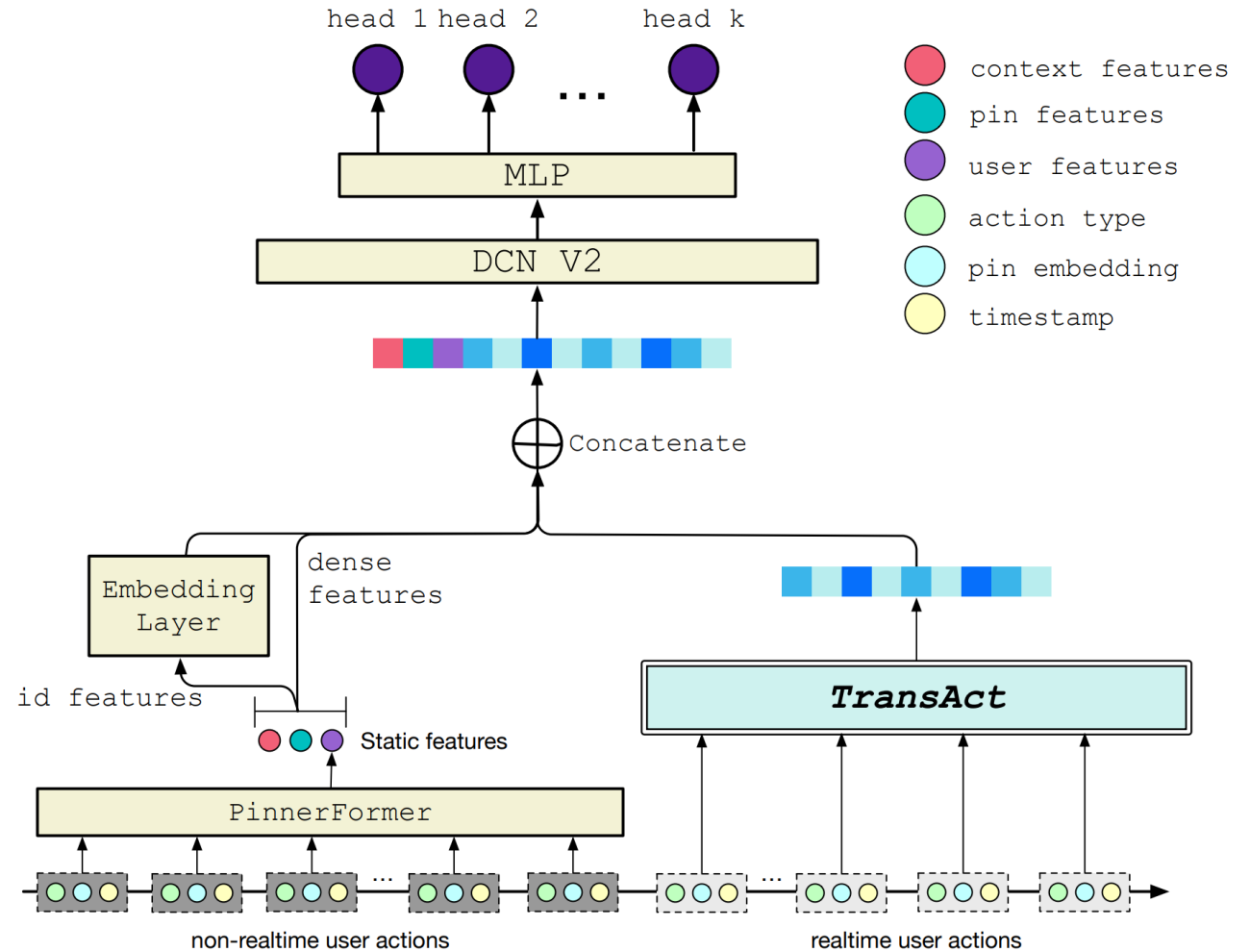
- Линейные модели и факторизационные машины
- Ранние архитектуры – Wide&Deep, DeepFM, DLRM
- DCN-v2
- Трансформеры над признаками – AutoInt, Hiformer, FAT, RankMixer

Sequential modeling

Вместо использования готовых пользовательских векторов можно учить анализ истории и основую ранжирующую модель **end-to-end**

- Подробнее в следующей лекции

А можно ли вообще убрать feature engineering? Про это – тоже дальше в курсе.

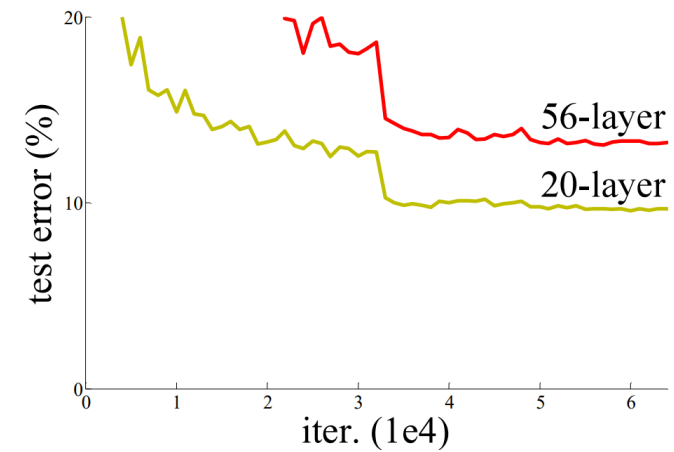
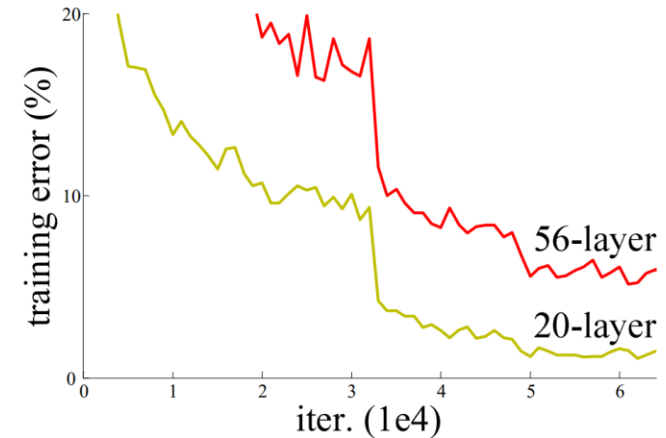


Is MLP enough?

Часть 2

Про MLP

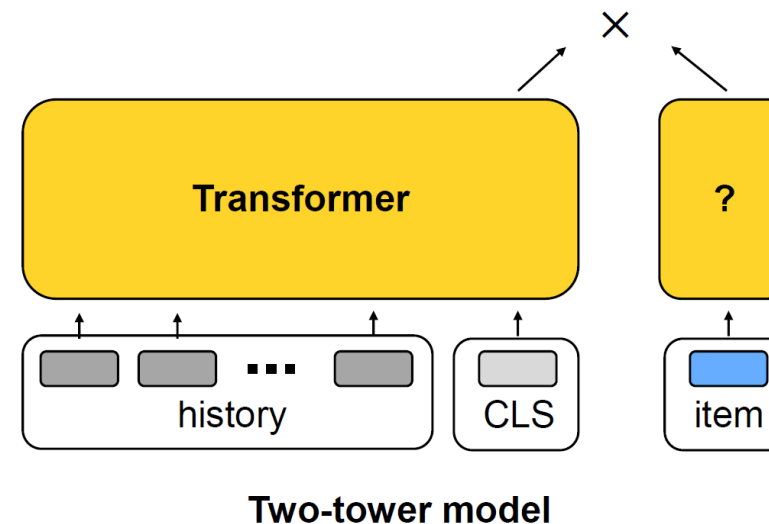
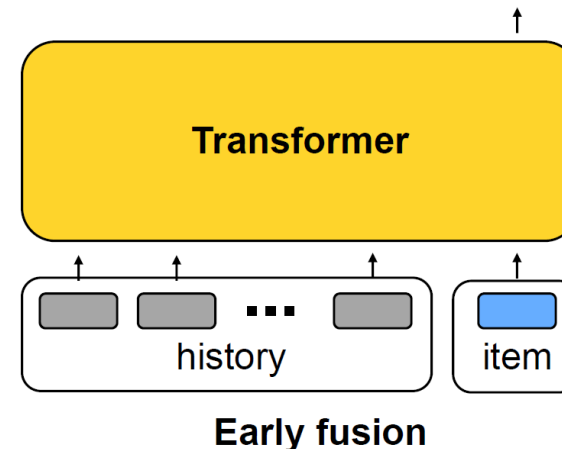
- MLP (многослойный персептрон) – полносвязная нейросеть из линейных слоев и функций активаций
- С увеличением количества слоев качество падает
- Градиенты затухают



История из практики

- Я делал первую двухбашенную трансформерную модель в Яндексе для рекомендаций
- Если в качестве башни айтема использовать MLP, сохраняем 50% прироста метрик
- Если заменить MLP на трансформер над одним токеном, то сохраняем уже 80% прироста

Что такое трансформер над одним токеном?

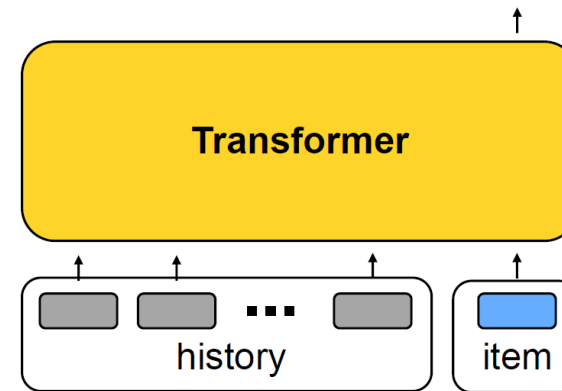


История из практики

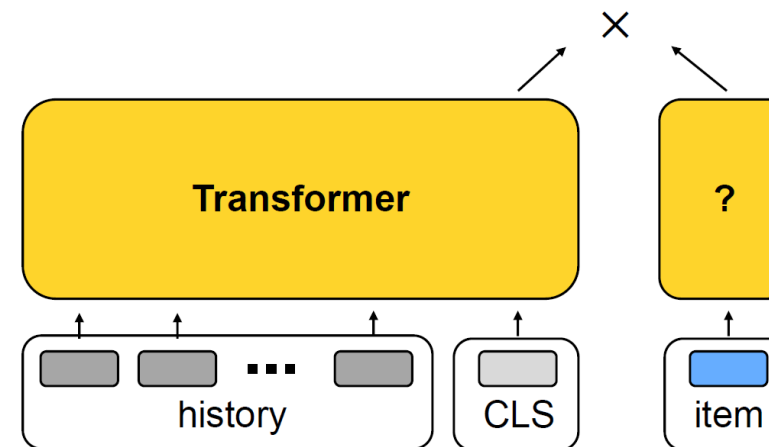
- Я делал первую двухбашенную трансформерную модель в Яндексе для рекомендаций
- Если в качестве башни айтема использовать MLP, сохраняем 50% прироста метрик
- Если заменить MLP на трансформер над одним токеном, то сохраняем уже 80% прироста

Что такое трансформер над одним токеном?

- Skip connections + layer normalization



Early fusion



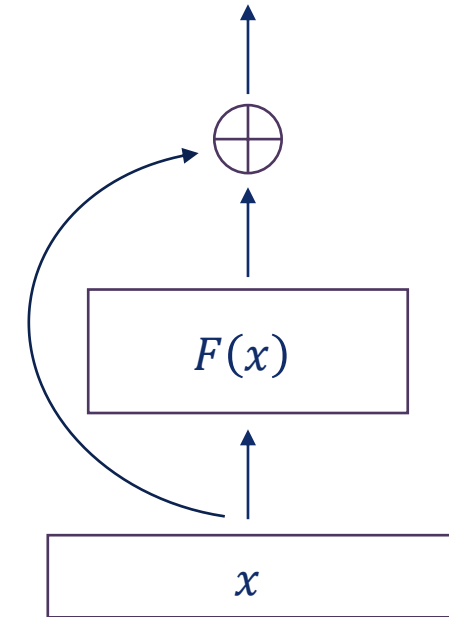
Two-tower model

ResNet

- Добавим после каждого слоя skip connection:

$$y = F(x) + x$$

- Решили проблему затухания градиентов
- Лишние слои можно занулить
- Обычно комбинируют с нормализацией



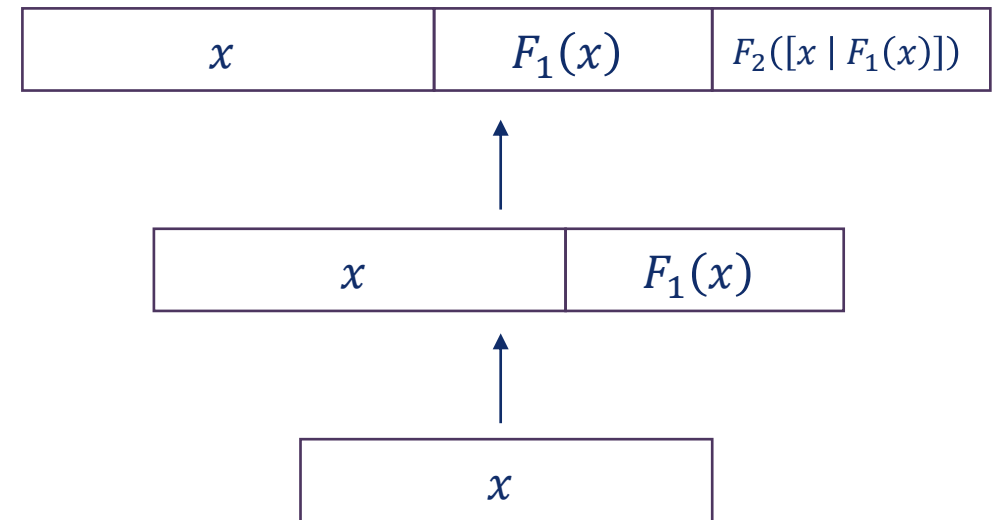
Проблема: вычислительно тяжелые слои из-за одинаковой размерности входов и выходов.

DenseNet

- Вместо суммы – конкатенация:

$$y = [x \mid F(x)]$$

- Функция $F(x)$ – дешёвый сужающий нейросетевой слой
- Каждый новый слой растит детализацию
- Модель ничего не забывает



Multi-task modeling

Часть 3

Multi-task learning

- В рекомендательных системах ранжирование учитывает сразу несколько сигналов
- На примере YouTube:
 - Engagement – клики, просмотры
 - Satisfaction – лайки, рейтинги
 - Long-term сигналы типа LTV (lifetime value)
- Если учить отдельные модели для каждого сигнала, это требует в разы больше ресурсов
 - А ещё, сложнее улучшать и поддерживать модели

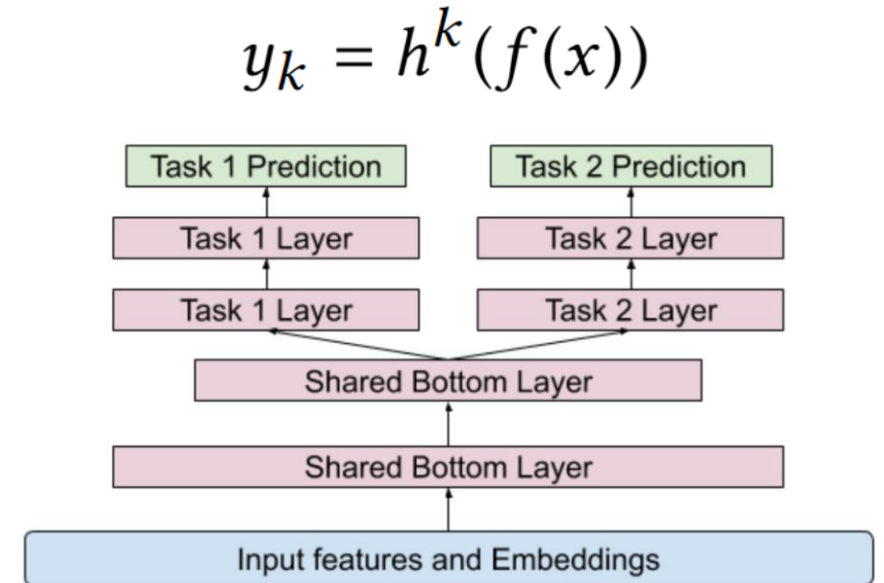
Shared bottom architecture

- Сначала применяем общие shared bottom слои
- Затем task-specific слои
- Плюсы: регуляризация, transfer learning, требуется меньше ресурсов

Но есть нюансы:

- Если есть конфликтующие задачи, это мешает обучению

Потенциальное решение – просто увеличить емкость модели.



(a) Shared-Bottom Model with shared bottom hidden layers and separate towers for two tasks.

[Recommending what video to watch next: a multitask ranking system](#)

Mixture of experts

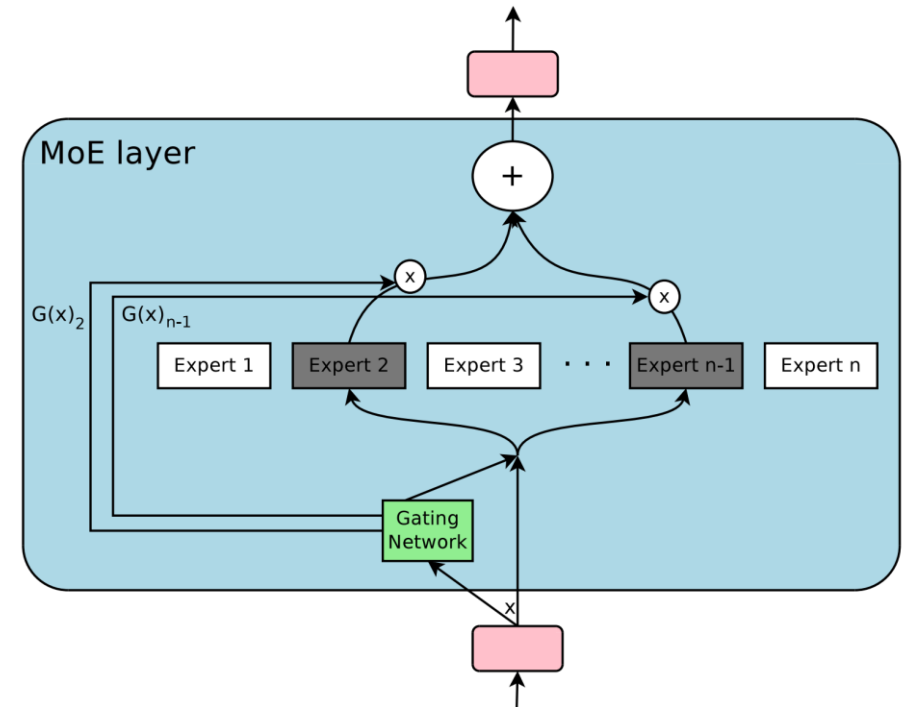
Изначальная MoE модель выглядела следующим образом:

$$y = \sum_{i=1}^n g(x)_i f_i(x), \quad \sum_{i=1}^n g(x)_i = 1$$

где $f_i(x)$ – это эксперты, а $g(x)$ - gating network, задающий веса экспертов.

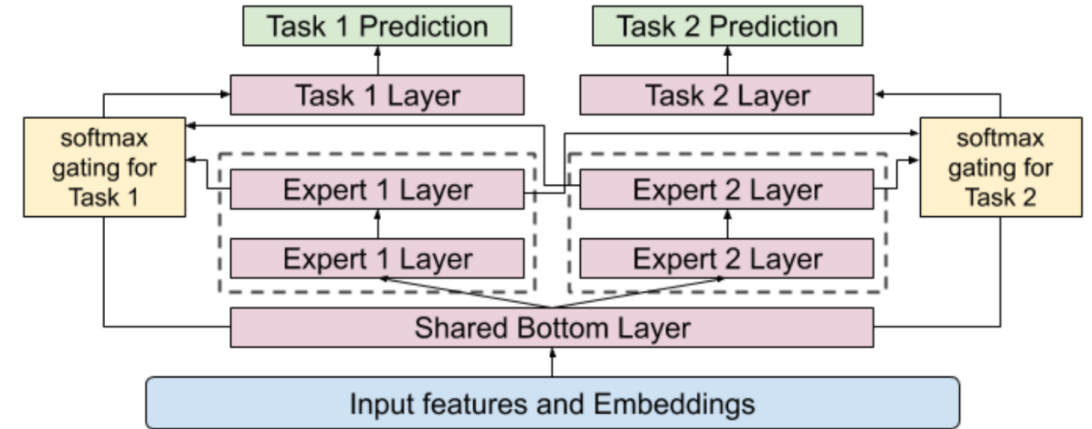
Затем стали делать отдельные MoE слои:

- Conditional computation – на каждом входном объекте участвует только часть экспертов



Multi-gate mixture of experts

- Заменяли shared bottom на смесь экспертов
 - В ММоЕ статье целиком, в YouTube статье – только часть
- У каждой задачи свой gating network, то есть свои веса экспертов
- Если задачи не связаны, то под каждую задачу можно выучить свои подмножества экспертов



[Modeling Task Relationships in Multi-task Learning with Multi-gate Mixture-of-Experts](#)

[Recommending what video to watch next: a multitask ranking system](#)

Multi-gate mixture of experts

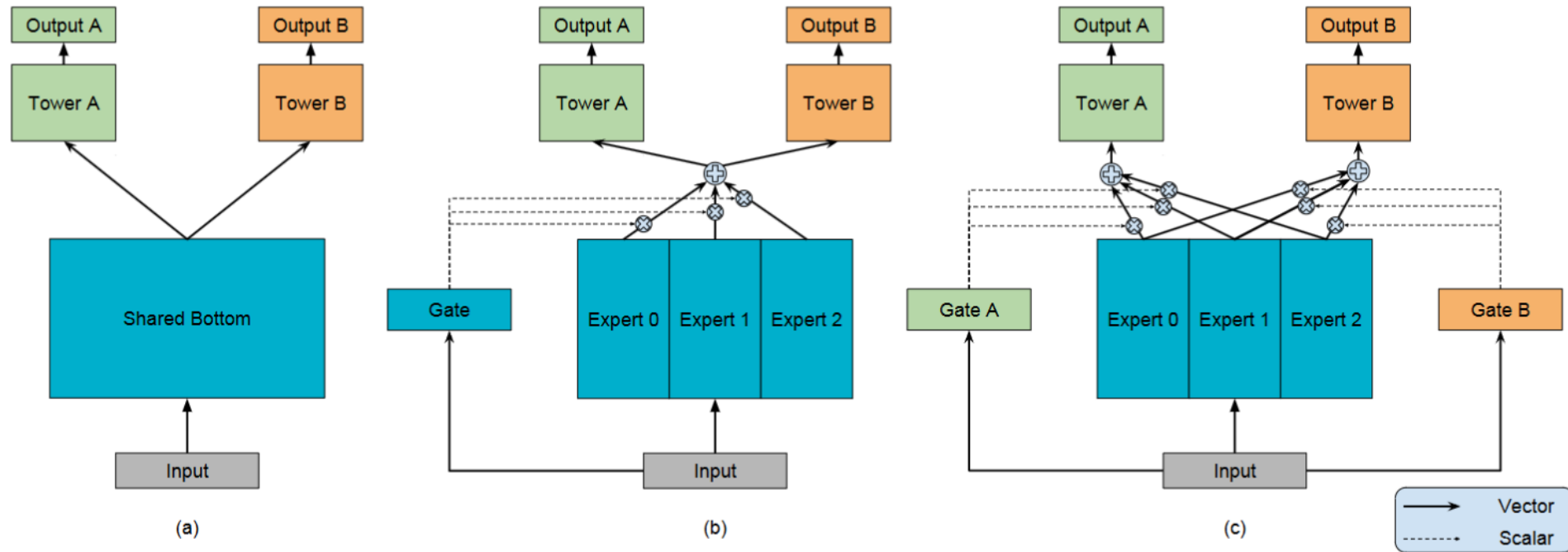


Figure 1: (a) Shared-Bottom model. (b) One-gate MoE model. (c) Multi-gate MoE model.

Case study: YouTube ranking

Сравнивают:

- **Single-Task:** учимся и ранжируем по engagement
- **Shared-Bottom** и **MMoE** (не объясняют как ранжируют)

Live experiment	Engagement Metric	Satisfaction Metric
Shared-Bottom Improvement over Single-Task	-0.22% *	19.72% **
MMoE Improvement over Shared-Bottom	0.25% **	2.65% **

* indicates confidence interval level 90%

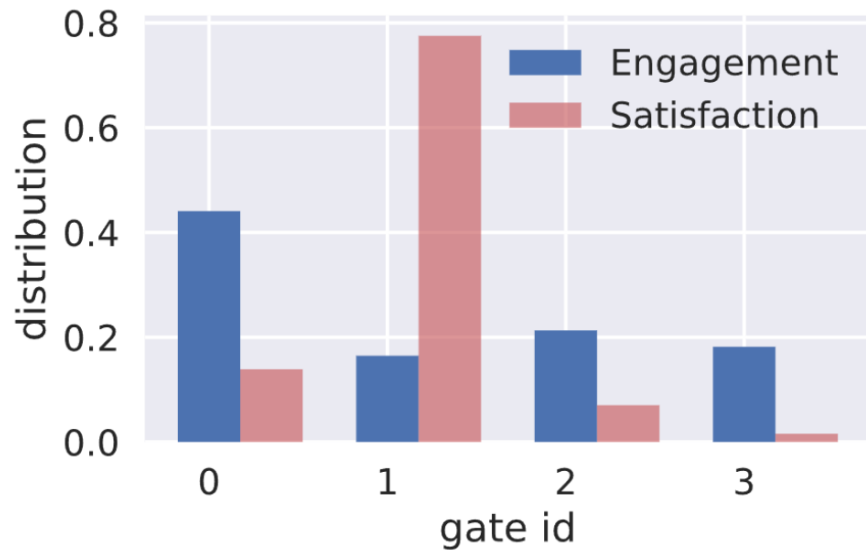
** indicates confidence interval level 95%

[Modeling Task Relationships in Multi-task Learning with Multi-gate Mixture-of-Experts](#)

Model Architecture	Number of Multiplications	Engagement Metric	Satisfaction Metric
Shared-Bottom	3.7M	/	/
Shared-Bottom	6.1M	+0.1%	+ 1.89%
MMoE (4 experts)	3.7M	+0.20%	+ 1.22%
MMoE (8 Experts)	6.1M	+0.45%	+ 3.07%

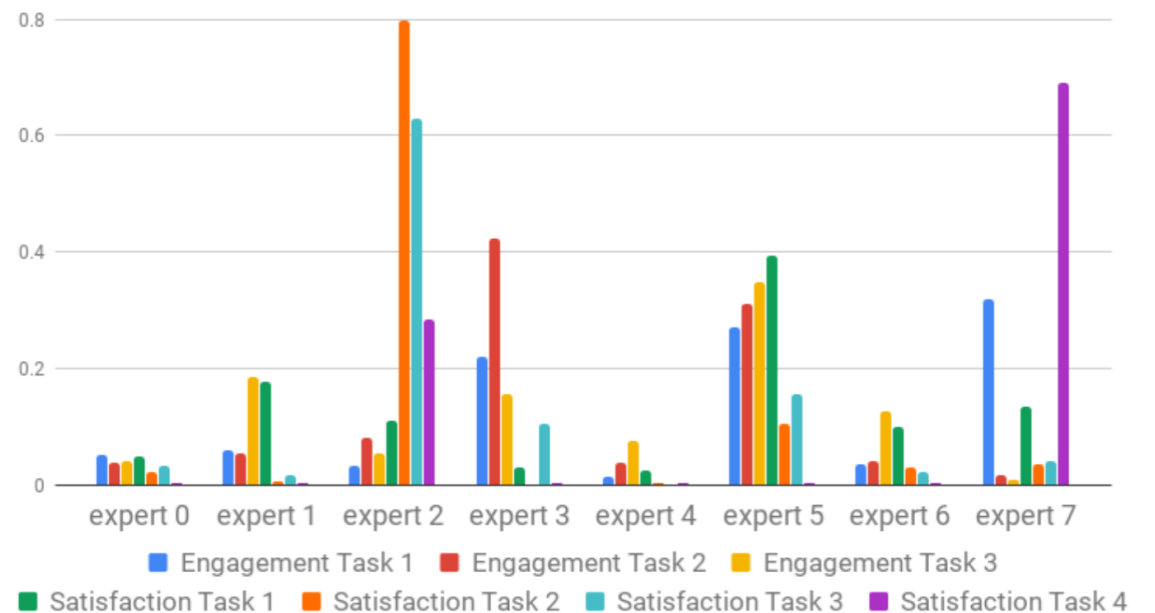
[Recommending what video to watch next: a multitask ranking system](#)

Case study: YouTube ranking



[Modeling Task Relationships in Multi-task Learning with Multi-gate Mixture-of-Experts](#)

Expert Utilization for Multiple Tasks



[Recommending what video to watch next: a multitask ranking system](#)

Появляются эксперты, которые специализируются на конкретных задачах.

Entire space multi-task modeling

- Пусть хотим ранжировать выдачу по покупкам

Entire space multi-task modeling

- Пусть хотим ранжировать выдачу по покупкам
- Надо учить модель на $P(\text{purchase} \mid \text{impression})$
- Не получается выучить такую модель – почему?

Entire space multi-task modeling

- Пусть хотим ранжировать выдачу по покупкам
- Надо учить модель на $P(\text{purchase} \mid \text{impression})$
- Не получается выучить такую модель – почему?
 - Data sparsity – очень мало позитивов, большой дисбаланс классов
 - Сложность сигнала – знаем мало информации, чтобы хорошо предсказывать покупки (много неизвестной нам информации превращается в шум)

Entire space multi-task modeling

- Давайте моделировать $P(\text{purchase} \mid \text{click})$
- Можно ли по такой модели делать ранжирование?
 - По-хорошему нет – в выдаче далеко не все айтемы будут кликнуты, а мы ранжируем так, будто все айтемы являются кликами
 - Несоответствие между обучением и применением

Entire space multi-task modeling

- Наша вероятностная модель:
 $impression \rightarrow click \rightarrow purchase$
- $P(purchase | impression) = P(click | impression) \cdot P(purchase | click)$
- Но вторая голова всё еще применяется не на том срезе, на котором учится
- Будем учить вторую голову на лосс по $P(purchase | impression)$

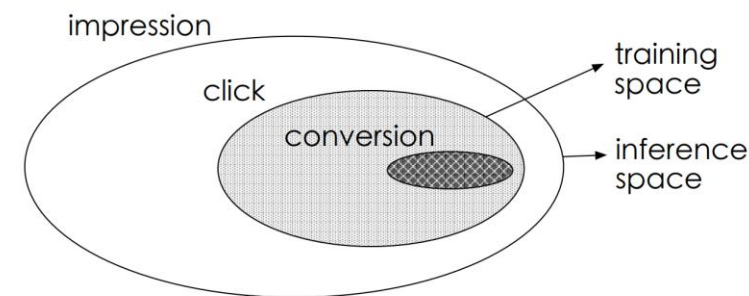
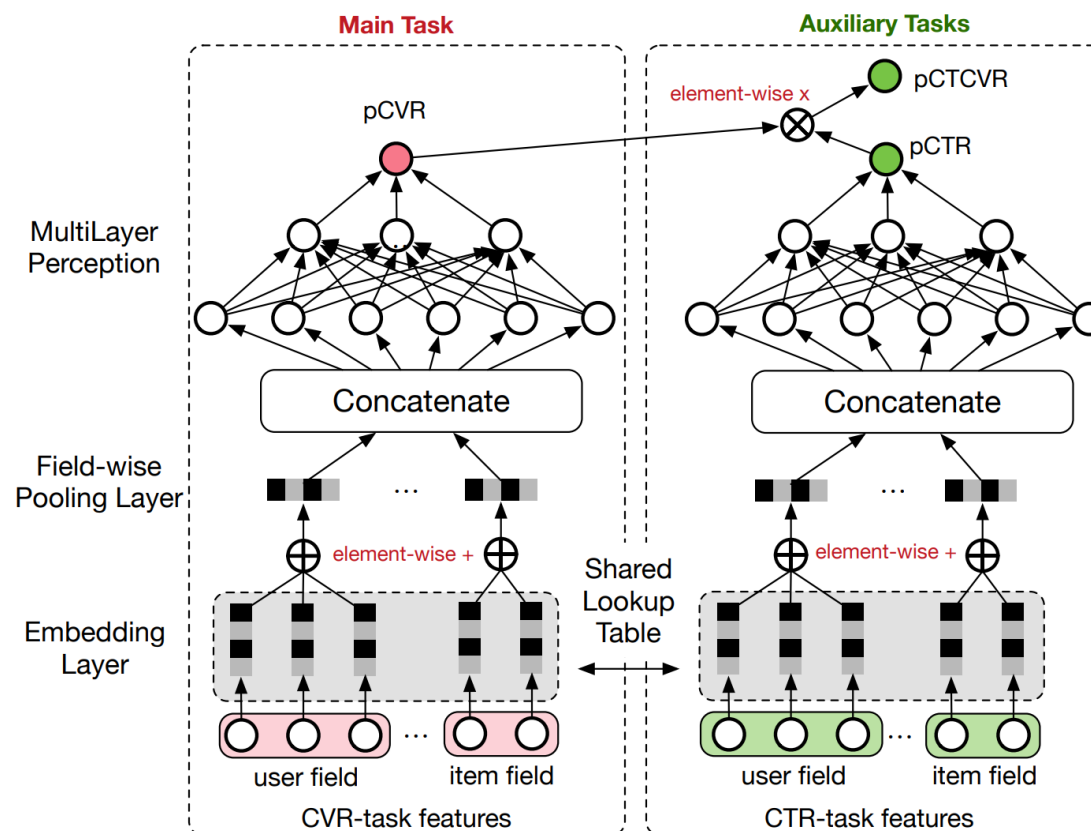


Figure 1: Illustration of sample selection bias problem in conventional CVR modeling. Training space is composed of samples with clicked impressions. It is only part of the inference space which is composed of all impressions.

Entire space multi-task modeling

- В рекламе решается похожая задача – conversion modelling
 - конверсии – почти то же самое, что покупки
- Нужно предсказывать клики и конверсии для аукциона
- Конверсии используются чтобы корректировать ставки и в аукционах, в которых платят за конверсии (вместо кликов)



[Entire Space Multi-Task Model: An Effective Approach for Estimating Post-Click Conversion Rate](#)

Position debiasing

Часть 4

Position bias

- На ранних позициях в выдаче CTR выше

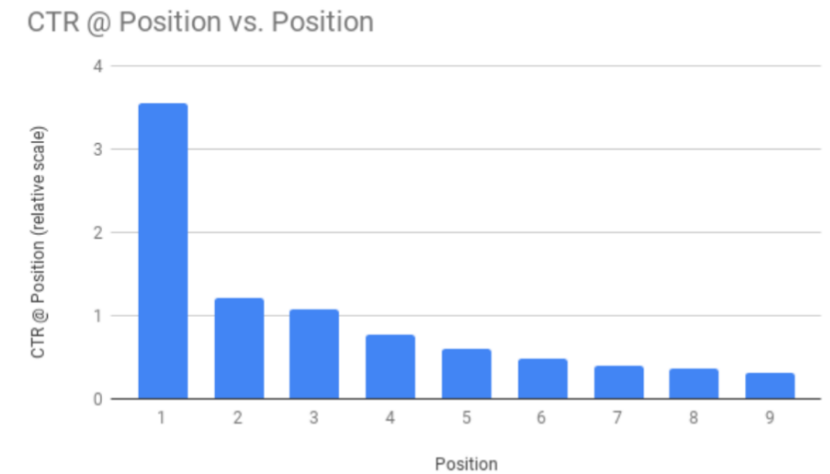


Figure 6: CTR for position 1 to 9.

Position bias

- На ранних позициях в выдаче CTR выше
 - Пользователи чаще кликают на ранние позиции (видят их раньше, а еще доверяют им больше)
 - На более высоких позициях более релевантные айтемы
- **Position bias:** есть false negatives – часть некликнутых айтемов тоже хороши, но их не увидели или увидели поздно
- **Feedback loop effect:** вверху выдачи то, что нравится системе; из-за position bias оно чаще получает положительный фидбек

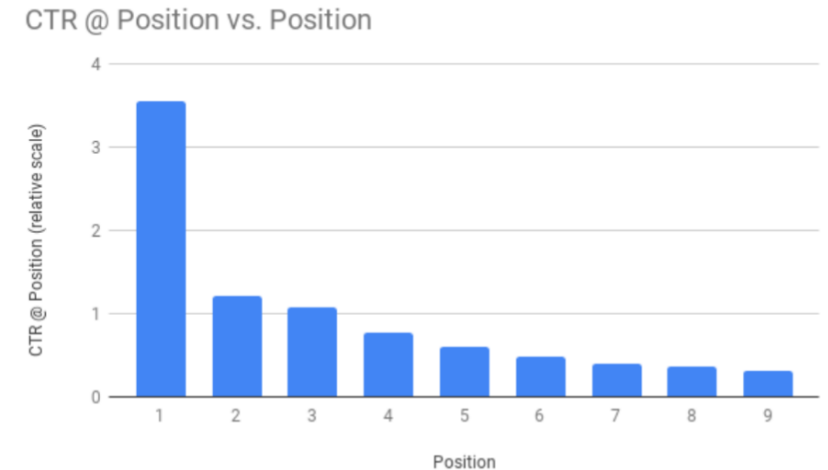


Figure 6: CTR for position 1 to 9.

Как с этим бороться

1. Учить модель на $P(\text{click} \mid \text{impression}, \text{pos} = 1)$, и качество оценивать также
2. Добавить признак позиции в модель, при применении подставить "pos = 1"
3. **Inversed propensity weighting:** при обучении перевзвесить сэмплы исходя из позиции
 - Но для этого нужно знать истинные CTR позиций (для случайного трафика)
4. Сделать отдельную bias башню

Shallow bias tower

- На входе в bias башню – вся контекстная информация, полезная для моделирования
 - e.g. device сильно влияет на position bias
- При обучении складываем выходы целевой модели и башни
 - Делаем дропаут выхода башни
 - Чтобы эта башня не перетягивала на себя сигнал релевантности
- На инференсе башню выкидываем

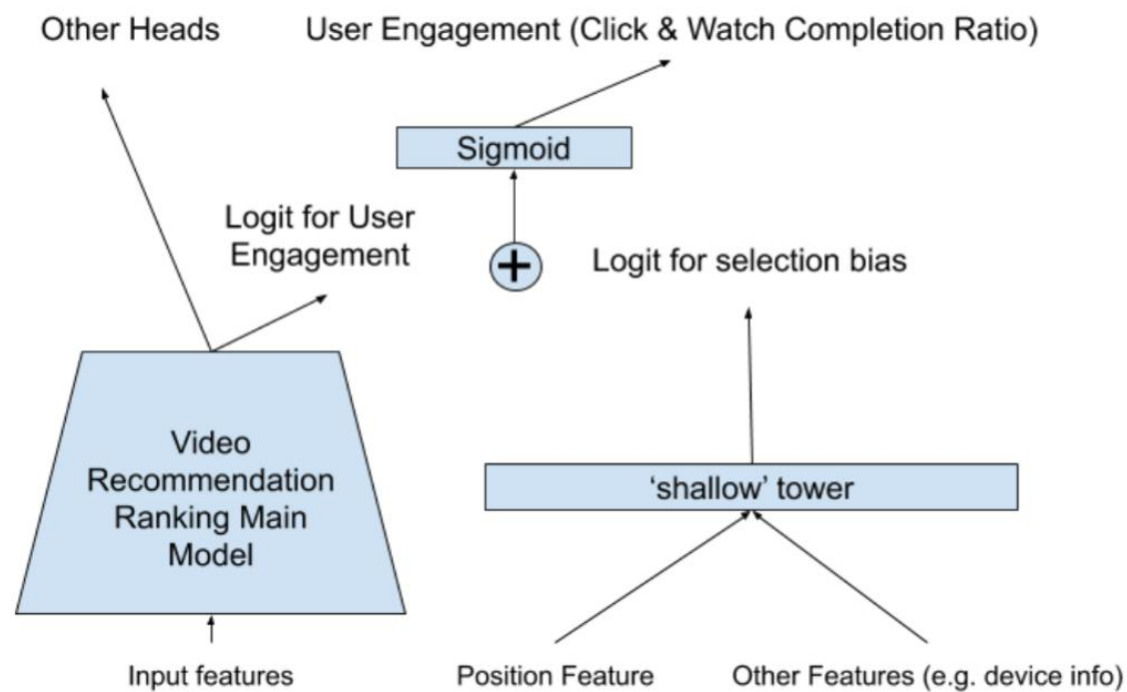


Figure 3: Adding a shallow side tower to learn selection bias (e.g., position bias).

Knowledge distillation

Часть 5

Knowledge distillation

- Accuracy-latency tradeoff:
 - Увеличиваем модель, получаем прирост качества
 - Но замедляемся
- Дистилляция:
 - Есть большая модель – **teacher model**
 - Хотим мощную небольшую модель – **student model**
 - **Direct distillation:** ученик учится на целевые метки и на предсказания учителя

Case study: YouTube ranking

- Предложили auxiliary distillation – у ученика две головы:
 - Одна учится на целевые метки, другая на предсказания учителя
 - Для ранжирования используется голова с целевыми метками
- У учителя есть свои bias'ы, не хотим, чтобы они влияли на ранжирование
- В некоторых задачах нужны откалиброванные предсказания (например, в рекламе)

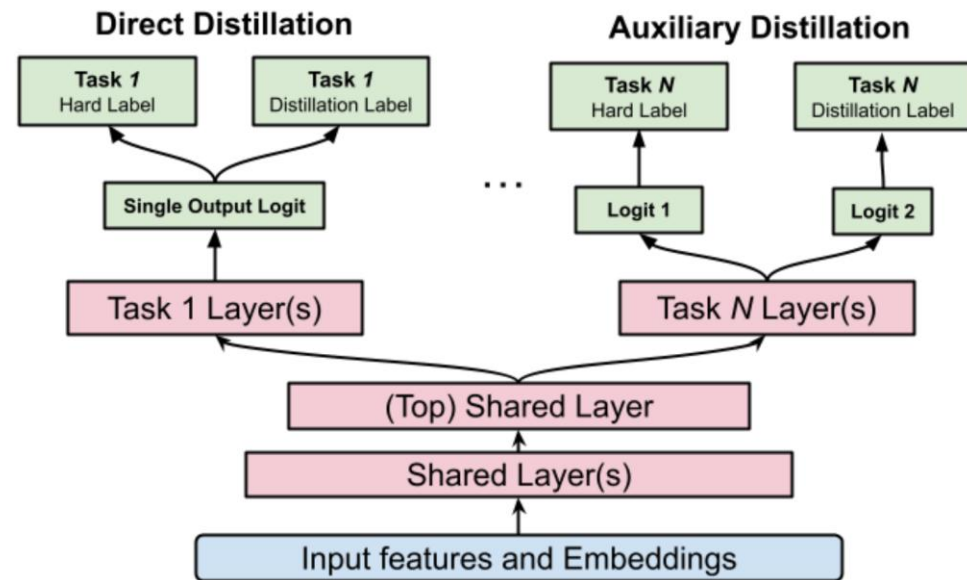


Figure 2: Direct vs Auxiliary distillation

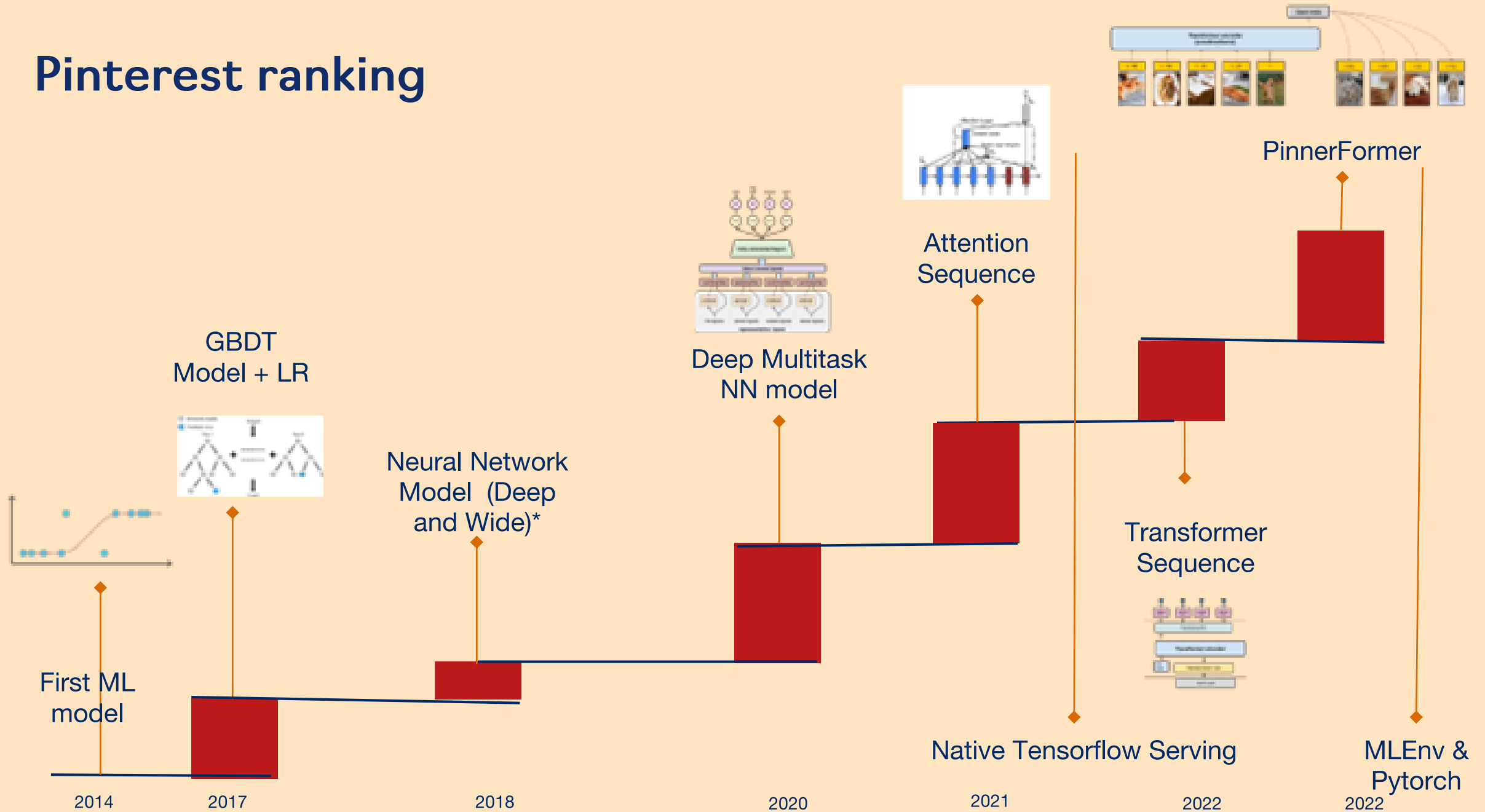
Выводы

Обсудили:

- Feature interaction layers
- MLP layers
- Multi-task modelling
- Position debiasing
- Knowledge distillation

На семинаре – разбор статьи про DCN-v2.

Pinterest ranking



Нейросетевое ранжирование

